

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторные работы
по курсу «Численные методы»
Вариант №2**

Выполнила: Былькова К. А.

Группа: М8О-408Б-22

Преподаватель: Ревизников Д. Л.

Оценка:

Дата: 08.12.25

Лабораторная работа № 5 (1)

Задание: используя явную и неявную конечно-разностные схемы, а также схему Кранка-Николсона, решить начально-краевую задачу для дифференциального уравнения параболического типа. Осуществить реализацию трех вариантов аппроксимации граничных условий, содержащих производные: двухточечная аппроксимация с первым порядком, трехточечная аппроксимация со вторым порядком, двухточечная аппроксимация со вторым порядком. В различные моменты времени вычислить погрешность численного решения путем сравнения результатов с приведенным в задании аналитическим решением $U(x, t)$.

2.

$$\frac{\partial u}{\partial t} = a \frac{\partial^2 u}{\partial x^2}, \quad a > 0,$$

$$u(0, t) = 0,$$

$$u(1, t) = 1,$$

$$u(x, 0) = x + \sin(\pi x).$$

Аналитическое решение: $U(x, t) = x + \exp(-\pi^2 at) \sin(\pi x)$

Описание решения:

Данное уравнение представляет собой уравнение теплопроводности с граничными условиями первого рода.

Явная конечно-разностная схема имеет вид:

$$\frac{u_j^{k+1} - u_j^k}{\tau} = a^2 \frac{u_{j+1}^k - 2u_j^k + u_{j-1}^k}{h^2}$$

Неявная:

$$\frac{u_j^{k+1} - u_j^k}{\tau} = a^2 \frac{u_{j+1}^{k+1} - 2u_j^{k+1} + u_{j-1}^{k+1}}{h^2}$$

Схема Кранка-Николсона:

$$\frac{u_j^{k+1} - u_j^k}{\tau} = \theta a^2 \frac{u_{j+1}^{k+1} - 2u_j^{k+1} + u_{j-1}^{k+1}}{h^2} + (1-\theta) a^2 \frac{u_{j+1}^k - 2u_j^k + u_{j-1}^k}{h^2}$$

где θ – вес явной части конечно-разностной схемы. Для нахождения неявной части конечно-разностной трехдиагональной матрицы. схемы применялся метод прогонки.

Погрешности вычислялись как модуль разности точного и вычисленного решения.

Текст программы:

```
import numpy as np
import matplotlib.pyplot as plt

a = 1.0

# Граничные условия
def phi0(t):
    return 0.0

def phi1(t):
    return 1.0

# Начальное условие
def init_cond(x):
    return x + np.sin(np.pi * x)

# Аналитическое решение
def analytical_solution(x, t, a=1.0):
    return x + np.exp(-np.pi**2 * a * t) * np.sin(np.pi * x)

# Метод прогонки (для трёхдиагональной системы)
def tma(a_diag, b_diag, c_diag, d_vec):
    n = len(b_diag)
    p = np.zeros(n)
    q = np.zeros(n)

    # Прямой ход
    p[0] = -c_diag[0] / b_diag[0]
    q[0] = d_vec[0] / b_diag[0]

    for i in range(1, n):
        denom = b_diag[i] + a_diag[i] * p[i - 1]
        p[i] = -c_diag[i] / denom
        q[i] = (d_vec[i] - a_diag[i] * q[i - 1]) / denom

    # Обратный ход
    x = np.zeros(n)
    x[-1] = q[-1]
    for i in range(n - 2, -1, -1):
        x[i] = p[i] * x[i + 1] + q[i]

    return x

def explicit(a, h, tau, t_range, x_range):
    t_start, t_end = t_range
    N = int(round((x_range[1] - x_range[0]) / h)) + 1
    time_steps = int(np.ceil((t_end - t_start) / tau)) + 1

    x = np.linspace(x_range[0], x_range[1], N)
    u = np.zeros((time_steps, N))
    u[0, :] = init_cond(x)

    sigma = a * tau / h**2
    if sigma > 0.5:
        print(f"Внимание: явная схема неустойчива! sigma = {sigma:.3f} > 0.5")

    for k in range(1, time_steps):
        for j in range(1, N - 1):
            u[k][j] = sigma * (u[k - 1][j + 1] + u[k - 1][j - 1]) + (1 - 2 * sigma) * u[k - 1][j]

        t_k = t_start + k * tau
        u[k][0] = phi0(t_k)
        u[k][-1] = phi1(t_k)
```

```

return u, x

def implicit(a, h, tau, t_range, x_range):
    t_start, t_end = t_range
    N = int(round((x_range[1] - x_range[0]) / h)) + 1
    time_steps = int(np.ceil((t_end - t_start) / tau)) + 1

    x = np.linspace(x_range[0], x_range[1], N)
    u = np.zeros((time_steps, N))
    u[0, :] = init_cond(x)

    sigma = a * tau / h**2

    for k in range(1, time_steps):
        a_diag = np.zeros(N)
        b_diag = np.zeros(N)
        c_diag = np.zeros(N)
        d_vec = np.zeros(N)

        # Внутренние узлы
        for j in range(1, N - 1):
            a_diag[j] = -sigma
            b_diag[j] = 1.0 + 2.0 * sigma
            c_diag[j] = -sigma
            d_vec[j] = u[k - 1, j]

        # Граничные условия
        t_k = t_start + k * tau
        b_diag[0] = 1.0
        c_diag[0] = 0.0
        d_vec[0] = phi0(t_k)

        a_diag[-1] = 0.0
        b_diag[-1] = 1.0
        d_vec[-1] = phi1(t_k)

        u[k, :] = tma(a_diag, b_diag, c_diag, d_vec)

    return u, x

def crank_nicolson(a, h, tau, t_range, x_range):
    t_start, t_end = t_range
    N = int(round((x_range[1] - x_range[0]) / h)) + 1
    time_steps = int(np.ceil((t_end - t_start) / tau)) + 1

    x = np.linspace(x_range[0], x_range[1], N)
    u = np.zeros((time_steps, N))
    u[0, :] = init_cond(x)

    sigma = a * tau / h**2

    for k in range(1, time_steps):
        a_diag = np.zeros(N)
        b_diag = np.zeros(N)
        c_diag = np.zeros(N)
        d_vec = np.zeros(N)

        for j in range(1, N - 1):
            a_diag[j] = -sigma / 2.0
            b_diag[j] = 1.0 + sigma
            c_diag[j] = -sigma / 2.0
            d_vec[j] = (1.0 - sigma) * u[k - 1, j] + (sigma / 2.0) * (u[k -
1, j - 1] + u[k - 1, j + 1])

        t_k = t_start + k * tau
        b_diag[0] = 1.0
        c_diag[0] = 0.0
        d_vec[0] = phi0(t_k)

```

```

        a_diag[-1] = 0.0
        b_diag[-1] = 1.0
        d_vec[-1] = phi1(t_k)

        u[k, :] = tma(a_diag, b_diag, c_diag, d_vec)

    return u, x

def plot_solutions(x, t_target, t_range, h, sigma, a, **solutions):
    tau = sigma * h**2 / a
    t_grid = np.arange(t_range[0], t_range[1] + tau, tau)
    k_target = np.argmin(np.abs(t_grid - t_target))

    plt.figure(figsize=(12, 6))
    for name, sol in solutions.items():
        plt.plot(x, sol[k_target, :], 'o', label=name)

    u_analytical = analytical_solution(x, t_target, a)
    plt.plot(x, u_analytical, 'k--', label='Analytical')

    plt.xlabel('x')
    plt.ylabel('u(x, t)')
    plt.title(f'Solutions at t = {t_target:.3f}')
    plt.legend()
    plt.grid(True)
    plt.show()

def plot_error_vs_time(x, t_grid, a, **solutions):
    plt.figure(figsize=(10, 6))

    for name, u_num in solutions.items():
        errors = []
        for k, t in enumerate(t_grid):
            u_true = analytical_solution(x, t, a)
            err = np.max(np.abs(u_num[k, :] - u_true))
            errors.append(err)
        plt.plot(t_grid, errors, label=name, linewidth=2)

    plt.xlabel('Time $t$')
    plt.ylabel('Max absolute error')
    plt.title('Max error vs time')
    plt.grid(True, which="both", ls="--")
    plt.legend()
    plt.show()

def main():
    a = 1.0
    x_range = [0.0, 1.0]
    t_range = [0.0, 0.5]

    N, K = 10, 125
    tau = (t_range[1] - t_range[0]) / K
    h = (x_range[1] - x_range[0]) / N

    # Проверка устойчивости явной схемы
    sigma = a * tau / h**2
    if sigma > 0.5:
        print('Явная схема неустойчива!')
        return 1

    # Все три схемы с одинаковыми h, tau
    u_exp, x = explicit(a, h, tau, t_range, x_range)
    u_imp, _ = implicit(a, h, tau, t_range, x_range)
    u_cn, _ = crank_nicolson(a, h, tau, t_range, x_range)

    # Согласуем количество временных шагов
    min_steps = min(u_exp.shape[0], u_imp.shape[0], u_cn.shape[0])
    u_exp = u_exp[:min_steps]
    u_imp = u_imp[:min_steps]
    u_cn = u_cn[:min_steps]

```

```

t_grid = np.linspace(t_range[0], t_range[0] + (min_steps - 1) * tau,
min_steps)

# Графики решений
for t_target in [0.05, 0.1, 0.25]:
    plot_solutions(
        x, t_target, t_range, h, sigma, a,
        Explicit=u_exp,
        Implicit=u_imp,
        Crank_Nicolson=u_cn
    )

# График ошибок
plot_error_vs_time(
    x, t_grid, a,
    Explicit=u_exp,
    Implicit=u_imp,
    Crank_Nicolson=u_cn
)

def compute_error_at_final_time(u_num, x, t_final, a):
    u_true = analytical_solution(x, t_final, a)
    return np.max(np.abs(u_num[-1, :] - u_true))

def convergence_in_time(a, h_fixed, t_range, tau_values, x_range):
    methods = {
        'Explicit': explicit,
        'Implicit': implicit,
        'Crank-Nicolson': crank_nicolson
    }
    errors = {name: [] for name in methods}
    valid_taus = []

    t_final = t_range[1]

    for tau in tau_values:
        # Проверка устойчивости явной схемы
        sigma = a * tau / h_fixed**2
        valid_taus.append(tau)

        for name, solver in methods.items():
            if name == 'Explicit' and sigma > 0.5:
                errors[name].append(np.nan)
                continue

            try:
                u, x = solver(a, h_fixed, tau, t_range, x_range)
                err = compute_error_at_final_time(u, x, t_final, a)
                errors[name].append(err)
            except Exception as e:
                print(f"Ошибка в {name} при tau={tau:.2e}: {e}")
                errors[name].append(np.nan)

    return np.array(valid_taus), {k: np.array(v) for k, v in errors.items()}

def convergence_in_space(a, sigma_fixed, t_range, h_values, x_range):
    methods = {
        'Explicit': explicit,
        'Implicit': implicit,
        'Crank-Nicolson': crank_nicolson
    }
    errors = {name: [] for name in methods}
    valid_hs = []

    t_final = t_range[1]

    for h in h_values:
        tau = sigma_fixed * h**2 / a
        sigma = a * tau / h**2

```

```

valid_hs.append(h)

for name, solver in methods.items():
    if name == 'Explicit' and sigma > 0.5:
        errors[name].append(np.nan)
        continue

    try:
        u, x = solver(a, h, tau, t_range, x_range)
        err = compute_error_at_final_time(u, x, t_final, a)
        errors[name].append(err)
    except Exception as e:
        print(f"Ошибка в {name} при h={h:.2e}, tau={tau:.2e}: {e}")
        errors[name].append(np.nan)

return np.array(valid_hs), {k: np.array(v) for k, v in errors.items()}

def plot_convergence(taus, err_tau_dict, hs, err_h_dict):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    colors = {
        'Explicit': 'red',
        'Implicit': 'blue',
        'Crank-Nicolson': 'green'
    }
    markers = {
        'Explicit': 'o',
        'Implicit': 's',
        'Crank-Nicolson': '^'
    }

    # Сходимость по времени
    for method in ['Explicit', 'Implicit', 'Crank-Nicolson']:
        err = err_tau_dict[method]
        valid = ~np.isnan(err)
        if np.any(valid):
            ax1.plot(taus[valid], err[valid],
                    marker=markers[method], color=colors[method],
                    label=method, markersize=6, linewidth=1.5)

    # Опорные линии
    #  $O(\tau)$  – для явной и неявной
    valid_imp = ~np.isnan(err_tau_dict['Implicit'])
    if np.any(valid_imp):
        tau_ref = taus[valid_imp]
        err_ref = err_tau_dict['Implicit'][valid_imp]
        c1 = err_ref[0] / tau_ref[0]
        ax1.plot(tau_ref, c1 * tau_ref, 'k--', label=r'$O(\tau)$')

    #  $O(\tau^2)$  – для Кранка-Николсона
    valid_cn = ~np.isnan(err_tau_dict['Crank-Nicolson'])
    if np.any(valid_cn):
        tau_cn = taus[valid_cn]
        err_cn = err_tau_dict['Crank-Nicolson'][valid_cn]
        c2 = err_cn[0] / (tau_cn[0] ** 2)
        ax1.plot(tau_cn, c2 * tau_cn**2, 'k-.', label=r'$O(\tau^2)$')

    ax1.set_xlabel(r'Time step $\tau$')
    ax1.set_ylabel('Max error at final time')
    ax1.set_title('Convergence in time (fixed $h$)')
    ax1.grid(True, which="both", ls=":", linewidth=0.5)
    ax1.legend()

    # Сходимость по пространству
    for method in ['Explicit', 'Implicit', 'Crank-Nicolson']:
        err = err_h_dict[method]
        valid = ~np.isnan(err)
        if np.any(valid):
            ax2.plot(hs[valid], err[valid],
                    marker=markers[method], color=colors[method],
                    label=method, markersize=6, linewidth=1.5)

```

```

# Опорная линия  $O(h^2)$ 
valid_cn_h = ~np.isnan(err_h_dict['Crank-Nicolson'])
if np.any(valid_cn_h):
    h_ref = hs[valid_cn_h]
    err_ref = err_h_dict['Crank-Nicolson'][valid_cn_h]
    c = err_ref[0] / (h_ref[0] ** 2)
    ax2.plot(h_ref, c * h_ref**2, 'k--', label=r'$O(h^2)$')

ax2.set_xlabel(r'Space step $h$')
ax2.set_ylabel('Max error at final time')
ax2.set_title('Convergence in space (fixed $tau$)')
ax2.grid(True, which="both", ls=":", linewidth=0.5)
ax2.legend()

plt.tight_layout()
plt.show()

def main_convergence():
    a = 1.0
    x_range = [0.0, 1.0]
    t_range = [0.0, 0.5]

    # Зависимость от шага по времени
    N_fixed = 10
    K_values = np.array([100, 125, 150, 200])
    h_fixed = (x_range[1] - x_range[0]) / N_fixed
    tau_values = (t_range[1] - t_range[0]) / K_values

    taus, err_tau_dict = convergence_in_time(a, h_fixed, t_range, tau_values,
x_range)

    # Зависимость от шага по пространству
    K_fixed = 125
    N_values = np.array([5, 10, 15, 20])
    tau_fixed = (t_range[1] - t_range[0]) / K_fixed
    h_values = (x_range[1] - x_range[0]) / N_values

    hs, err_h_dict = convergence_in_space(a, tau_fixed, t_range, h_values,
x_range)

    plot_convergence(taus, err_tau_dict, hs, err_h_dict)

if __name__ == "__main__":
    main()
    # main_convergence()

```


Результат работы программы:

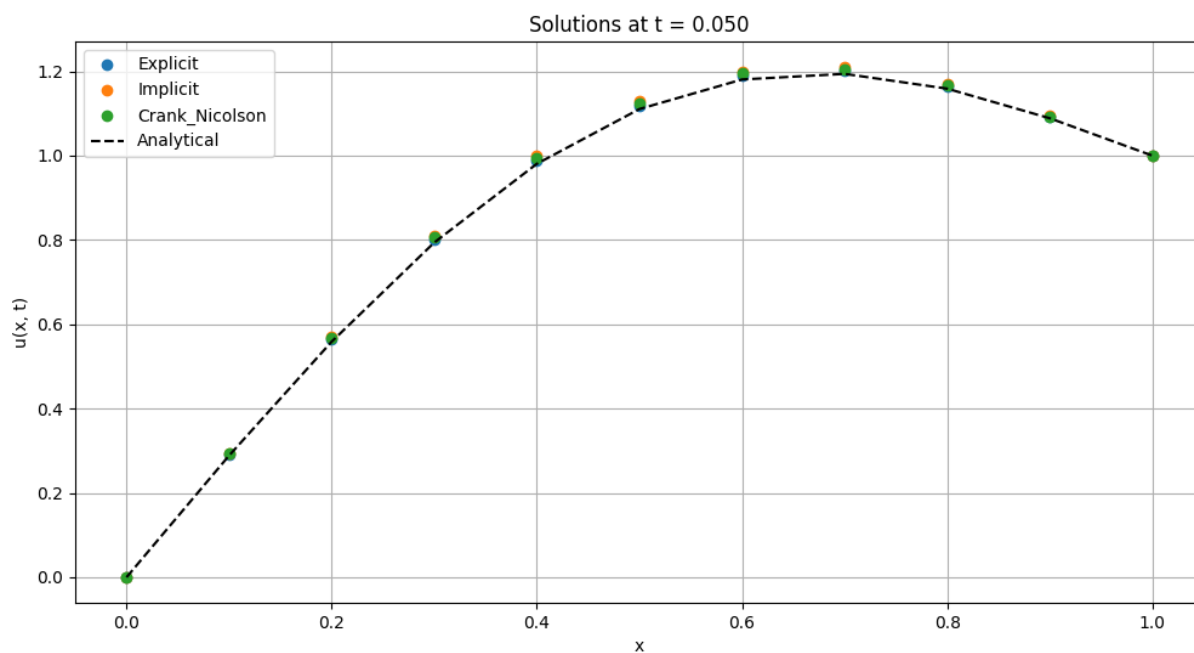


Рисунок 1. Сравнение численных решений с аналитическим в момент времени 0.05 секунд

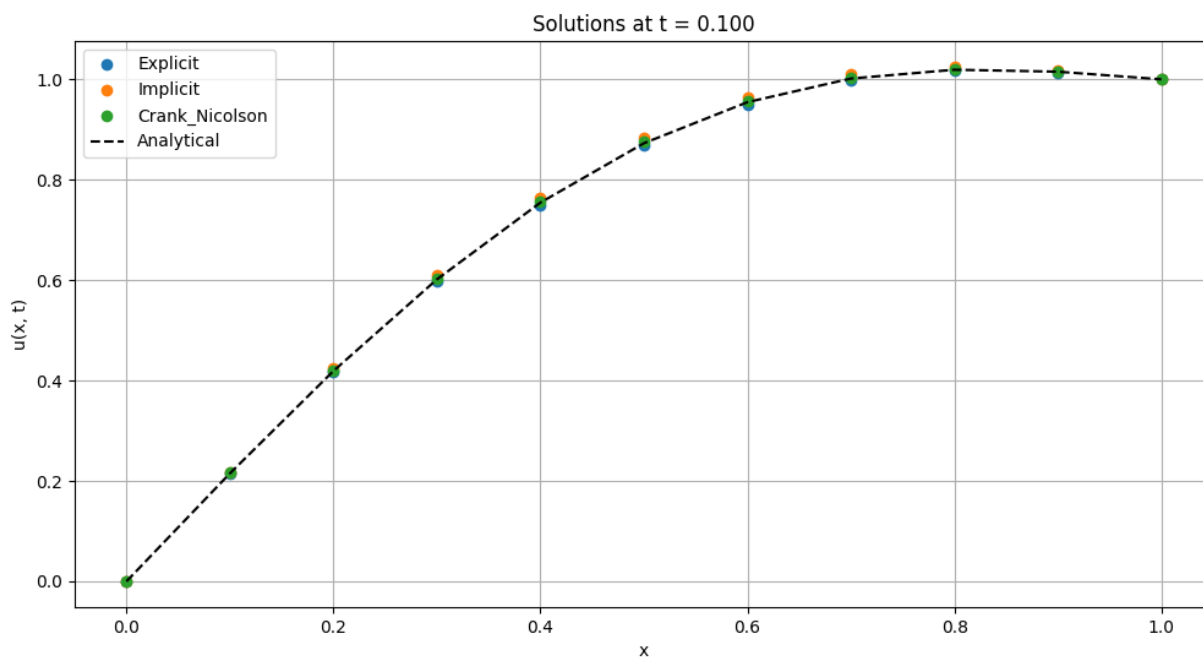


Рисунок 2. Сравнение численных решений с аналитическим в момент времени 0.1 секунда

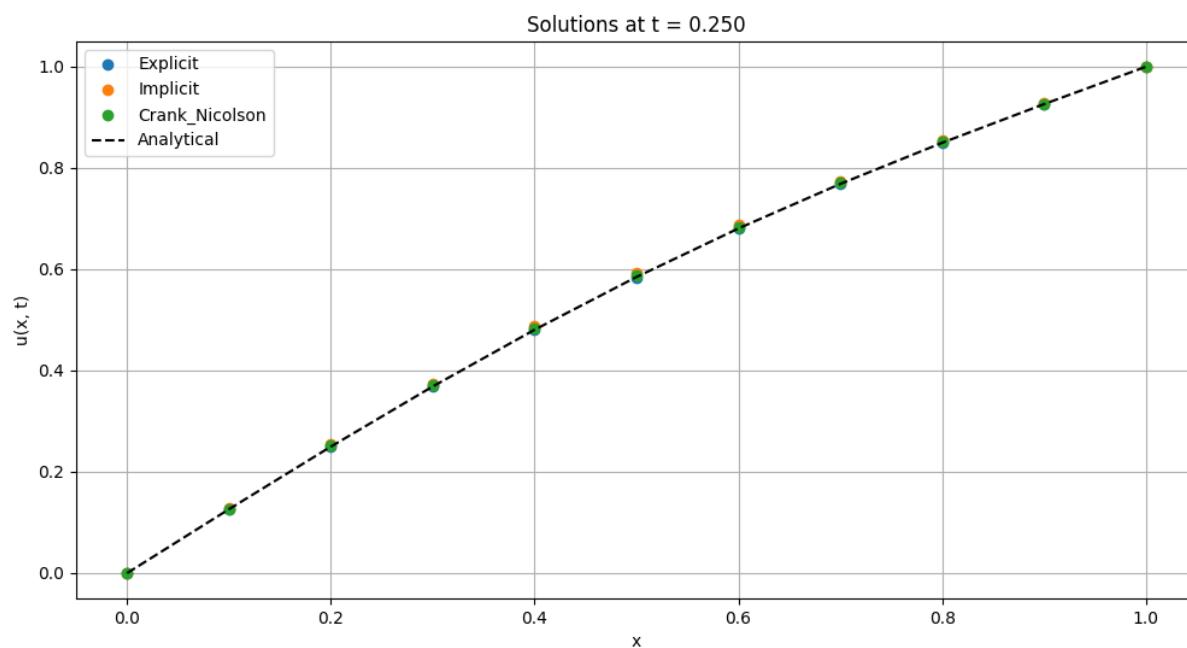


Рисунок 3. Сравнение численных решений с аналитическим в момент времени 0.25 секунд

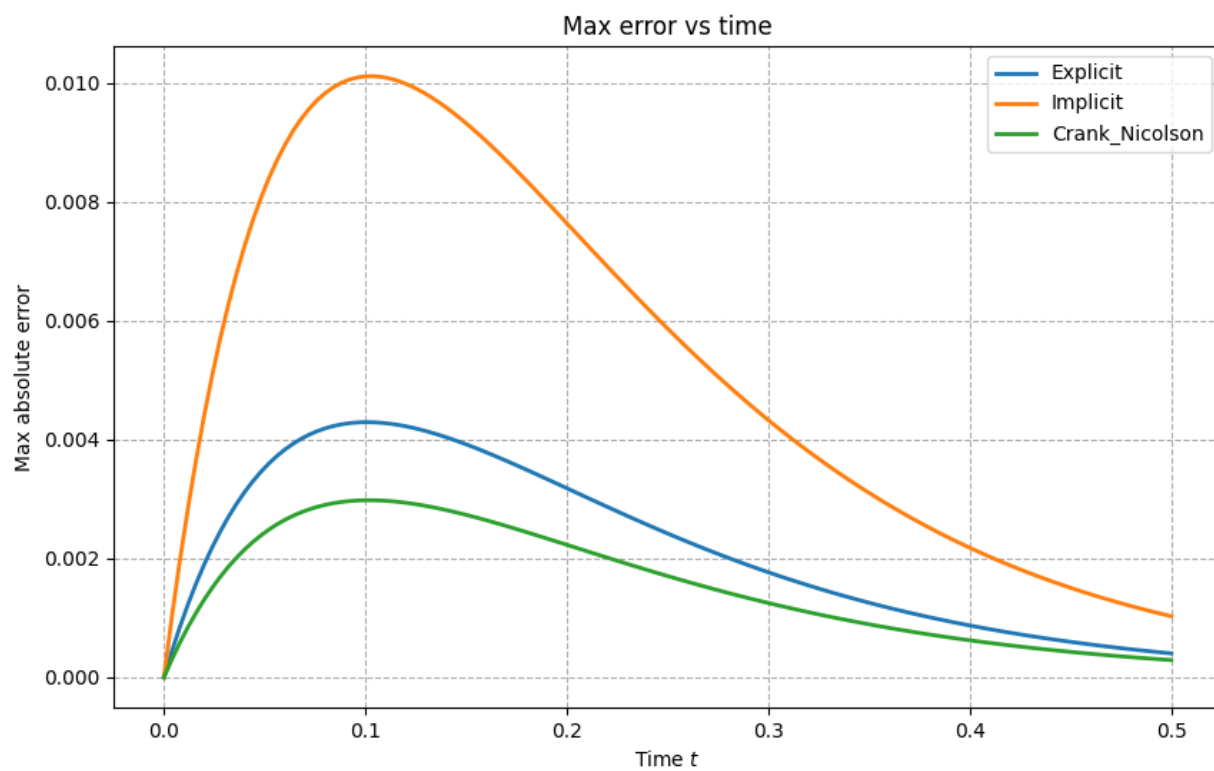


Рисунок 4. Зависимость погрешностей от времени

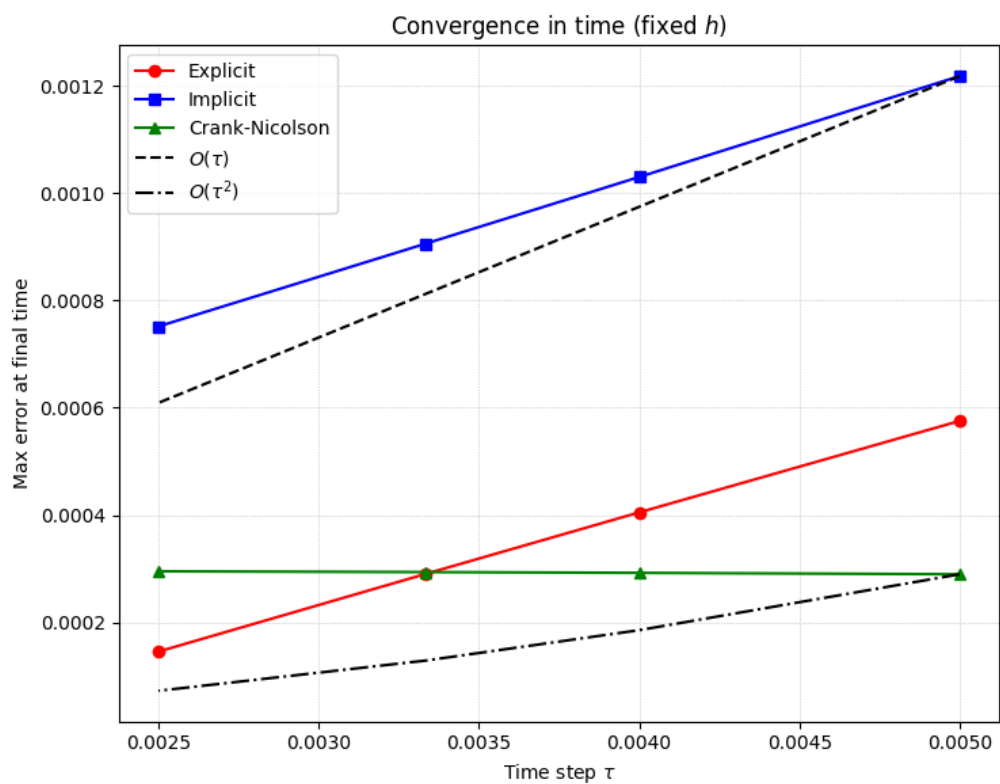


Рисунок 5. Зависимость погрешностей от шага по времени

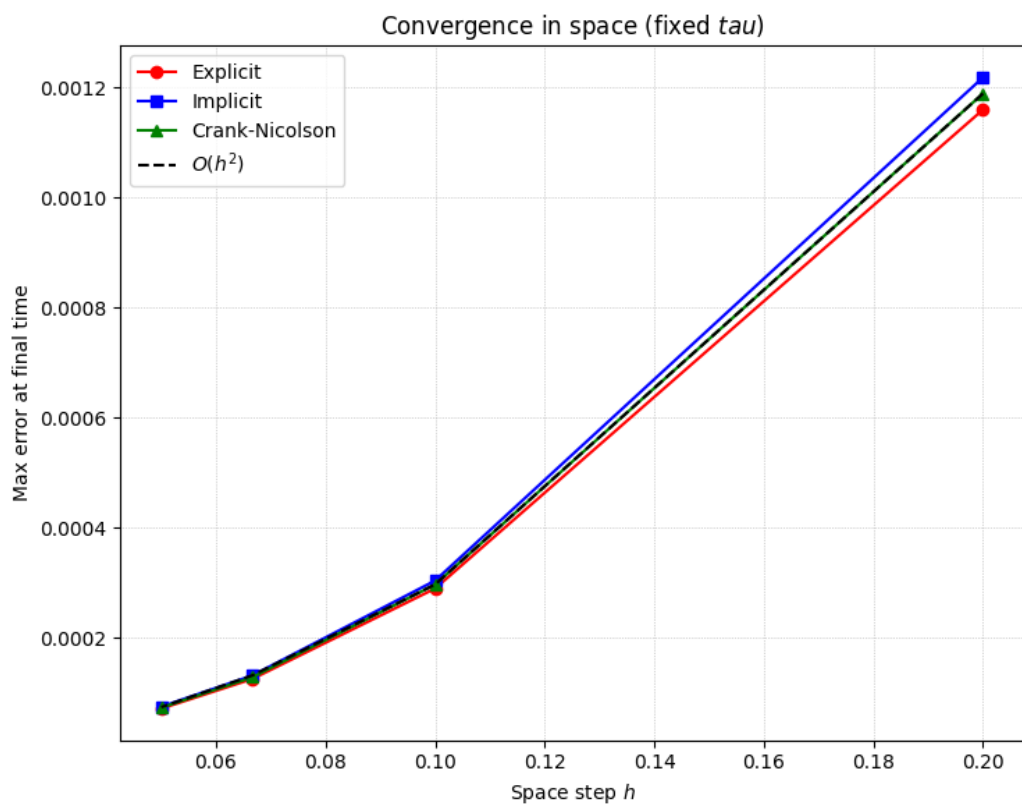


Рисунок 6. Зависимость погрешностей от шага по пространству

Лабораторная работа № 6 (2)

Задание: используя явную схему крест и неявную схему, решить начально-краевую задачу для дифференциального уравнения гиперболического типа. Аппроксимацию второго начального условия произвести с первым и со вторым порядком. Осуществить реализацию трех вариантов аппроксимации граничных условий, содержащих производные: двухточечная аппроксимация с первым порядком, трехточечная аппроксимация со вторым порядком, двухточечная аппроксимация со вторым порядком. В различные моменты времени вычислить погрешность численного решения путем сравнения результатов с приведенным в задании аналитическим решением $U(x, t)$.

2.

$$\frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2}, \quad a^2 > 0,$$

$$u_x(0, t) - u(0, t) = 0,$$

$$u_x(\pi, t) - u(\pi, t) = 0,$$

$$u(x, 0) = \sin x + \cos x,$$

$$u_t(x, 0) = -a(\sin x + \cos x).$$

Аналитическое решение: $U(x, t) = \sin(x - at) + \cos(x + at)$

Описание решения:

Явная конечно-разностная схема уравнения имеет вид:

$$\frac{u_j^{k+1} - 2u_j^k + u_j^{k-1}}{\tau^2} = a^2 \frac{u_{j+1}^k - 2u_j^k + u_{j-1}^k}{h^2}$$

Неявная:

$$\frac{u_j^{k+1} - 2u_j^k + u_j^{k-1}}{\tau^2} = a^2 \frac{u_{j+1}^{k+1} - 2u_j^{k+1} + u_{j-1}^{k+1}}{h^2}$$

Для аппроксимации начальных условий применяются два подхода, основанные на разложении в ряд Тейлора. Для второго начального условия $u_t(x, 0)$ используется разностная аппроксимация. При первом порядке точности применяется простая разностная аппроксимация

$$\frac{u_j^1 - u_j^0}{\tau} = -a(\sin x_j + \cos x_j)$$

Для достижения второго порядка точности используется разложение в ряд Тейлора с учетом исходного дифференциального уравнения:

$$u_j^1 = u_j^0 + \tau \cdot u_t(x_j, 0) + \frac{\tau^2}{2} \cdot u_{tt}(x_j, 0)$$

Из исходного уравнения вычисляем:

$$u_{tt}(x_j, 0) = a^2 u_{xx}(x, 0) = -a^2(\sin x + \cos x)$$

Итого:

$$u_j^1 = (\sin x_j + \cos x_j) - a\tau(\sin x_j + \cos x_j) - \frac{a^2\tau^2}{2}(\sin x_j + \cos x_j)$$

Для аппроксимации граничных условий третьего рода используются три различных метода.

Первый метод - двухточечная аппроксимация с первым порядком, основанная на разностной аппроксимации производной:

$$\begin{aligned} \frac{u_1^{k+1} - u_0^{k+1}}{h} - u_0^{k+1} &= 0 \Rightarrow u_0^{k+1} = \frac{u_1^{k+1}}{1 + h} \\ \frac{u_n^{k+1} - u_{n-1}^{k+1}}{h} - u_n^{k+1} &= 0 \Rightarrow u_n^{k+1} = \frac{u_{n-1}^{k+1}}{1 - h} \end{aligned}$$

Второй метод - трехточечная аппроксимация со вторым порядком, использующая симметричные разностные шаблоны:

$$\begin{aligned} \frac{-3u_0^{k+1} + 4u_1^{k+1} - u_2^{k+1}}{2h} - u_0^{k+1} &= 0 \Rightarrow u_0^{k+1} = \frac{4u_1^{k+1} - u_2^{k+1}}{3 + 2h} \\ \frac{u_{N-2}^{k+1} - 4u_{N-1}^{k+1} + 3u_N^{k+1}}{2h} - u_N^{k+1} &= 0 \Rightarrow u_N^{k+1} = \frac{4u_{N-1}^{k+1} - u_{N-2}^{k+1}}{3 - 2h} \end{aligned}$$

Третий метод - двухточечная аппроксимация со вторым порядком, комбинирующая значения решения на разных временных слоях:

$$\begin{aligned} u_0^{k+1} &= \frac{[2(1 - \sigma)u_0^k + 2\sigma u_1^k - u_0^{k-1}]}{1 + 2\sigma h} \\ u_N^{k+1} &= \frac{[2(1 - \sigma)u_N^k + 2\sigma u_{N-1}^k - u_N^{k-1}]}{1 - 2\sigma h} \end{aligned}$$

Текст программы:

```
import numpy as np
import matplotlib.pyplot as plt

a = 1.0

# Граничные условия
u_0 = 0
u_1 = 0

# Начальные условия
def init_cond(x):
    return np.sin(x) + np.cos(x) # первое начальное условие

def d_init_cond(x, tau, a=1.0, method='first_order'):
    u0 = init_cond(x)
    ut0 = -a * (np.sin(x) + np.cos(x)) # второе начальное условие

    if method == 'first_order':
        # Аппроксимация 1-го порядка:  $u(x, \tau) = u(x, 0) + \tau * u_t(x, 0)$ 
        return u0 + tau * ut0
    elif method == 'second_order':
        # Аппроксимация 2-го порядка:  $u(x, \tau) = u(x, 0) + \tau * u_t(x, 0) + (\tau^2/2) * u_{tt}(x, 0)$ 
        #  $u_{tt}(x, 0) = a^2 * u_{xx}(x, 0)$ 
        uxx0 = -np.sin(x) - np.cos(x) # вторая производная  $u(x, 0)$ 
        utt0 = a**2 * uxx0
        return u0 + tau * ut0 + 0.5 * tau**2 * utt0

# Аналитическое решение
def analytical_solution(x, t, a=1.0):
    return np.sin(x - a * t) + np.cos(x + a * t)

# Метод прогонки (для трёхдиагональной системы)
def tma(a, b):
    n = len(b)
    p = np.zeros(n)
    q = np.zeros(n)
    # Прямой ход
    p[0] = -a[0][1] / a[0][0]
    q[0] = b[0] / a[0][0]

    for i in range(1, len(p) - 1):
        p[i] = -a[i][i + 1] / (a[i][i] + a[i][i - 1] * p[i - 1])
        q[i] = (b[i] - a[i][i - 1] * q[i - 1]) / (a[i][i] + a[i][i - 1] * p[i - 1])
    i = len(a) - 1
    p[-1] = 0
    q[-1] = (b[-1] - a[-1][-2] * q[-2]) / (a[-1][-1] + a[-1][-2] * p[-2])

    # Обратный ход
    x = np.zeros(n)
    x[-1] = q[-1]
    for i in range(n - 2, -1, -1):
        x[i] = p[i] * x[i + 1] + q[i]
    return x

def apply_boundary_conditions(u_current, h, method='two_point_first_order'):
    N = len(u_current)

    if method == 'two_point_first_order':
        # Двухточечная аппроксимация 1-го порядка
```

```

# u_x = (u[1] - u[0])/h = u[0] => u[0] = u[1]/(1 + h)
u_current[0] = u_current[1] / (1 + h)
# u_x = (u[N-1] - u[N-2])/h = u[N-1] => u[N-1] = u[N-2]/(1 -
h)
u_current[N-1] = u_current[N-2] / (1 - h)

elif method == 'three_point_second_order':
    # Трехточечная аппроксимация 2-го порядка
    # u_x = (-3u[0] + 4u[1] - u[2])/(2h) = u[0]
    # => u[0] = (4u[1] - u[2])/(3 + 2h)
    u_current[0] = (4 * u_current[1] - u_current[2]) / (3 + 2 * h)
    # u_x = (3u[N-1] - 4u[N-2] + u[N-3])/(2h) = u[N-1]
    # => u[N-1] = (4u[N-2] - u[N-3])/(3 - 2h)
    u_current[N-1] = (4 * u_current[N-2] - u_current[N-3]) / (3 -
2 * h)

elif method == 'two_point_second_order':
    # Двухточечная аппроксимация 2-го порядка
    u_current[0] = u_current[1] / (1 + h)
    u_current[N-1] = u_current[N-2] / (1 - h)

return u_current

def explicit(h, tau, t_range, x_range,
boundary_method='two_point_first_order', init_approx='first_order'):
    t_start, t_end = t_range
    N = int(round((x_range[1] - x_range[0]) / h)) + 1
    time_steps = int(np.ceil((t_end - t_start) / tau)) + 1

    x = np.linspace(x_range[0], x_range[1], N)
    u = np.zeros((time_steps, N))
    u[0, :] = init_cond(x)

    sigma = a**2 * tau**2 / h**2
    if sigma > 1:
        print(f"Внимание: явная схема неустойчива! sigma = {sigma:.3f}
> 1")

    # Второй временной слой с разной аппроксимацией начального условия
    for i in range(N):
        u[1][i] = d_init_cond(x[i], tau, a, init_approx)

    # Основной цикл по времени
    for k in range(2, time_steps):
        for j in range(1, N-1):
            u[k][j] = sigma * u[k - 1][j - 1] + 2 * (1 - sigma) * u[k
- 1][j] + sigma * u[k - 1][j + 1] - u[k - 2][j]

        # Применяем граничные условия выбранным методом
        u[k] = apply_boundary_conditions(u[k], h, boundary_method)

    return u, x, time_steps

def implicit(h, tau, t_range, x_range,
boundary_method='two_point_first_order', init_approx='first_order'):
    t_start, t_end = t_range
    N = int(round((x_range[1] - x_range[0]) / h)) + 1
    time_steps = int(np.ceil((t_end - t_start) / tau)) + 1

    x = np.linspace(x_range[0], x_range[1], N)
    u = np.zeros((time_steps, N))
    u[0, :] = init_cond(x)

    sigma = a**2 * tau**2 / h**2
    a_j = sigma
    b_j = -(1 + 2 * sigma)

```

```

c_j = sigma

# Второй временной слой
for i in range(N):
    u[1][i] = d_init_cond(x[i], tau, a, init_approx)

for k in range(2, time_steps):
    # Создаем матрицу для полной системы
    matrix = np.zeros((N, N))
    d = np.zeros(N)

    # Внутренние точки (j = 1 до N-2)
    for j in range(1, N-1):
        matrix[j][j-1] = a_j
        matrix[j][j] = b_j
        matrix[j][j+1] = c_j
        d[j] = -2 * u[k-1][j] + u[k-2][j]

    # Граничные условия
    if boundary_method == 'two_point_first_order':
        # Левая граница:  $u_x(0) = u(0) \Rightarrow (u_1 - u_0)/h = u_0 \Rightarrow u_0 = u_1/(1+h)$ 
        # В матричной форме:  $(1+h)u_0 - u_1 = 0$ 
        matrix[0][0] = 1 + h
        matrix[0][1] = -1
        d[0] = 0

        # Правая граница:  $u_x(\pi) = u(\pi) \Rightarrow (u_{N-1} - u_{N-2})/h = u_{N-1} \Rightarrow u_{N-1} = u_{N-2}/(1-h)$ 
        # В матричной форме:  $-u_{N-2} + (1-h)u_{N-1} = 0$ 
        matrix[N-1][N-2] = -1
        matrix[N-1][N-1] = 1 - h
        d[N-1] = 0

    elif boundary_method == 'three_point_second_order':
        # Левая граница:  $(-3u_0 + 4u_1 - u_2)/(2h) = u_0$ 
        #  $\Rightarrow -3u_0 + 4u_1 - u_2 = 2h*u_0$ 
        #  $\Rightarrow (-3-2h)u_0 + 4u_1 - u_2 = 0$ 
        matrix[0][0] = -3 - 2*h
        matrix[0][1] = 4
        matrix[0][2] = -1
        d[0] = 0

        # Правая граница:  $(3u_{N-1} - 4u_{N-2} + u_{N-3})/(2h) = u_{N-1}$ 
        #  $\Rightarrow 3u_{N-1} - 4u_{N-2} + u_{N-3} = 2h*u_{N-1}$ 
        #  $\Rightarrow u_{N-3} - 4u_{N-2} + (3-2h)u_{N-1} = 0$ 
        matrix[N-1][N-3] = 1
        matrix[N-1][N-2] = -4
        matrix[N-1][N-1] = 3 - 2*h
        d[N-1] = 0

    elif boundary_method == 'two_point_second_order':
        # Для двухточечной 2-го порядка используем тот же подход,
        # что и для 1-го порядка
        # так как симметричная разность требует фиктивных точек
        matrix[0][0] = 1 + h
        matrix[0][1] = -1
        d[0] = 0

        matrix[N-1][N-2] = -1
        matrix[N-1][N-1] = 1 - h
        d[N-1] = 0

    # Решаем полную систему
    try:
        u[k] = np.linalg.solve(matrix, d)

```



```

        except np.linalg.LinAlgError:
            # Если матрица вырождена, используем псевдообратную
            u[k] = np.linalg.lstsq(matrix, d, rcond=None)[0]

    return u, x, time_steps

def plot_solutions(x, t_target, t_grid, title_suffix='', **solutions):
    k_target = np.argmin(np.abs(t_grid - t_target))
    actual_time = t_grid[k_target]

    plt.figure(figsize=(12, 6))
    for name, sol in solutions.items():
        plt.plot(x, sol[k_target, :], 'o', label=name, markersize=4,
alpha=0.7)

    u_analytical = analytical_solution(x, actual_time, a)
    plt.plot(x, u_analytical, 'k-', label='Analytical', linewidth=2)

    plt.xlabel('x')
    plt.ylabel('u(x, t)')
    plt.title(f'Solutions at t = {t_target:.3f} {title_suffix}')
    plt.legend()
    plt.grid(True)
    plt.show()

def plot_error_vs_time(x, t_grid, a, title_suffix='', **solutions):
    plt.figure(figsize=(10, 6))

    for name, u_num in solutions.items():
        errors = []
        for k, t in enumerate(t_grid):
            u_true = analytical_solution(x, t, a)
            err = np.max(np.abs(u_num[k, :] - u_true))
            errors.append(err)
        plt.plot(t_grid[:len(errors)], errors, label=name,
linewidth=2)

    plt.xlabel('Time $t$')
    plt.ylabel('Max absolute error')
    plt.title(f'Max error vs time {title_suffix}')
    plt.grid(True, which="both", ls="--")
    plt.legend()
    plt.yscale('log')
    plt.show()

# Граничные условия: Двухточечная 1-го порядка
# Второе начальное условие: Аппроксимация 1-го порядка
def main():
    x_range = [0.0, np.pi]
    # t_range = [0.0, 2.0]
    t_range = [0.0, 1.0]

    K, N = 50, 40
    tau = (t_range[1] - t_range[0]) / K
    h = (x_range[1] - x_range[0]) / N

    sigma = a**2 * tau**2 / h**2
    # print(f'sigma = {sigma:.3f}')
    if sigma > 1:
        print('Явная схема неустойчива!')
        return 1

    u_exp, x, time_steps_exp = explicit(h, tau, t_range, x_range)
    u_imp, _, time_steps_imp = implicit(h, tau, t_range, x_range)

```

```

# Сопласуем количество временных шагов
min_steps = min(time_steps_exp, time_steps_imp)
u_exp = u_exp[:min_steps]
u_imp = u_imp[:min_steps]

t_grid = np.linspace(t_range[0], t_range[0] + (min_steps - 1) *
tau, min_steps)

# Графики решений
# for t_target in [1.0, 1.25, 1.5]:
for t_target in [0.1, 0.25, 0.5]:
    plot_solutions(
        x, t_target, t_grid,
        Explicit=u_exp,
        Implicit=u_imp
    )

# График ошибок
plot_error_vs_time(
    x, t_grid, a,
    Explicit=u_exp,
    Implicit=u_imp
)

# Сравнение методов с разной аппроксимацией граничных условий
def main_approx():
    x_range = [0.0, np.pi]
    # t_range = [0.0, 2.0]
    t_range = [0.0, 1.0]

    K, N = 50, 40
    tau = (t_range[1] - t_range[0]) / K
    h = (x_range[1] - x_range[0]) / N

    sigma = a**2 * tau**2 / h**2
    if sigma > 1:
        print('Явная схема неустойчива!')
        return 1

    boundary_methods = {
        'two_point_first_order': 'Двухточечная 1-го порядка (граничных условий)',
        'three_point_second_order': 'Трехточечная 2-го порядка (граничных условий)',
        'two_point_second_order': 'Двухточечная 2-го порядка (граничных условий)'
    }

    for method, description in boundary_methods.items():
        u_exp, x, time_steps_exp = explicit(h, tau, t_range, x_range,
boundary_method=method)
        u_imp, _, time_steps_imp = implicit(h, tau, t_range, x_range,
boundary_method=method)

        min_steps = min(time_steps_exp, time_steps_imp)
        u_exp = u_exp[:min_steps]
        u_imp = u_imp[:min_steps]

        t_grid = np.linspace(t_range[0], t_range[0] + (min_steps - 1)
* tau, min_steps)

        plot_solutions(
            # x, 1.5, t_grid, description,
            x, 0.5, t_grid, description,
            Explicit=u_exp,

```

```

        Implicit=u_imp
    )

    # plot_error_vs_time(
    #     x, t_grid, a, description,
    #     Explicit=u_exp,
    #     Implicit=u_imp
    # )

# Сравнение методов с разной аппроксимацией второго начального условия
def main_init_approx():
    x_range = [0.0, np.pi]
    # t_range = [0.0, 2.0]
    t_range = [0.0, 1.0]

    K, N = 50, 40
    tau = (t_range[1] - t_range[0]) / K
    h = (x_range[1] - x_range[0]) / N

    sigma = a**2 * tau**2 / h**2
    if sigma > 1:
        print('Явная схема неустойчива!')
        return 1

    init_methods = {
        'first_order': 'Аппроксимация 1-го порядка (2-го
нач. условия)',
        'second_order': 'Аппроксимация 2-го порядка (2-го
нач. условия)'
    }

    for method, description in init_methods.items():
        u_exp, x, time_steps_exp = explicit(h, tau, t_range, x_range,
boundary_method='two_point_first_order',
init_approx=method)
        u_imp, _, time_steps_imp = implicit(h, tau, t_range, x_range,
boundary_method='two_point_first_order',
init_approx=method)

        min_steps = min(time_steps_exp, time_steps_imp)
        u_exp = u_exp[:min_steps]
        u_imp = u_imp[:min_steps]

        t_grid = np.linspace(t_range[0], t_range[0] + (min_steps - 1)
* tau, min_steps)

        plot_solutions(
            # x, 1.5, t_grid, description,
            x, 0.5, t_grid, description,
            Explicit=u_exp,
            Implicit=u_imp
        )

        # plot_error_vs_time(
        #     x, t_grid, a, description,
        #     Explicit=u_exp,
        #     Implicit=u_imp
        # )

def compute_error_at_final_time(u_num, x, t_final, a):
    u_true = analytical_solution(x, t_final, a)
    return np.max(np.abs(u_num[-1, :] - u_true))

```

```

def convergence_in_time(a, h_fixed, t_range, tau_values, x_range):
    errors_explicit = []

    errors_implicit = []
    valid_taus = []

    t_final = t_range[1]

    for tau in tau_values:
        # Проверка устойчивости явной схемы
        sigma = a**2 * tau**2 / h_fixed**2

        if sigma > 1.0:
            print(f"tau={tau:.6f}: sigma={sigma:.3f} > 1, пропускаем
явную схему")
            errors_explicit.append(np.nan)
        else:
            try:
                u_exp, x, _ = explicit(h_fixed, tau, t_range, x_range)
                err_exp = compute_error_at_final_time(u_exp, x,
t_final, a)
                errors_explicit.append(err_exp)
            except Exception as e:
                print(f"Ошибка в явной схеме при tau={tau:.6f}: {e}")
                errors_explicit.append(np.nan)

            try:
                u_imp, x, _ = implicit(h_fixed, tau, t_range, x_range)
                err_imp = compute_error_at_final_time(u_imp, x, t_final,
a)
                errors_implicit.append(err_imp)
            except Exception as e:
                print(f"Ошибка в неявной схеме при tau={tau:.6f}: {e}")
                errors_implicit.append(np.nan)

            valid_taus.append(tau)

    return np.array(valid_taus), {'Explicit':
np.array(errors_explicit),
                                'Implicit':
np.array(errors_implicit)}

def convergence_in_space(a, tau_fixed, t_range, h_values, x_range):
    errors_explicit = []
    errors_implicit = []
    valid_hs = []

    t_final = t_range[1]

    for h in h_values:
        # Проверка устойчивости явной схемы
        sigma = a**2 * tau_fixed**2 / h**2

        if sigma > 1.0:
            print(f"h={h:.6f}: sigma={sigma:.3f} > 1, пропускаем явную
схему")
            errors_explicit.append(np.nan)
        else:
            try:
                u_exp, x, _ = explicit(h, tau_fixed, t_range, x_range)
                err_exp = compute_error_at_final_time(u_exp, x,
t_final, a)
                errors_explicit.append(err_exp)
            except Exception as e:
                print(f"Ошибка в явной схеме при h={h:.6f}: {e}")
                errors_explicit.append(np.nan)

```

```

        try:
            u_imp, x, _ = implicit(h, tau_fixed, t_range, x_range)
            err_imp = compute_error_at_final_time(u_imp, x, t_final,
a)
            errors_implicit.append(err_imp)
        except Exception as e:
            print(f"Ошибка в неявной схеме при h={h:.6f}: {e}")
            errors_implicit.append(np.nan)

        valid_hs.append(h)

    return np.array(valid_hs), {'Explicit': np.array(errors_explicit),
                                'Implicit':
np.array(errors_implicit)}

def plot_convergence(taus, err_tau_dict, hs, err_h_dict):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    colors = {
        'Explicit': 'red',
        'Implicit': 'blue'
    }
    markers = {
        'Explicit': 'o',
        'Implicit': 's'
    }

    # Сходимость по времени (при фиксированном h)
    for method in ['Explicit', 'Implicit']:
        err = err_tau_dict[method]
        valid = ~np.isnan(err)
        if np.any(valid):
            ax1.plot(taus[valid], err[valid],
                    marker=markers[method], color=colors[method],
                    label=method, markersize=6, linewidth=1.5)

    # Теоретические линии сходимости
    valid_imp = ~np.isnan(err_tau_dict['Implicit'])
    if np.any(valid_imp):
        tau_ref = taus[valid_imp]
        err_ref = err_tau_dict['Implicit'][valid_imp]
        C1 = err_ref[0] / tau_ref[0]
        ax1.plot(tau_ref, C1 * tau_ref, 'k--', label=r'$O(\tau)$')

    ax1.set_xlabel(r'Шаг по времени $\tau$')
    ax1.set_ylabel('Максимальная погрешность в конечный момент')
    ax1.set_title('Сходимость по времени (фиксированный $h$)')
    ax1.grid(True, which="both", ls=":", linewidth=0.5)
    ax1.set_xscale('log')
    ax1.set_yscale('log')
    ax1.legend()

    # Сходимость по пространству (при фиксированном tau)
    for method in ['Explicit', 'Implicit']:
        err = err_h_dict[method]
        valid = ~np.isnan(err)
        if np.any(valid):
            ax2.plot(hs[valid], err[valid],
                    marker=markers[method], color=colors[method],
                    label=method, markersize=6, linewidth=1.5)

    # Теоретическая линия сходимости $O(h^2)$
    valid_exp = ~np.isnan(err_h_dict['Explicit'])
    if np.any(valid_exp):
        h_ref = hs[valid_exp]

```

```

err_ref = err_h_dict['Explicit'][valid_exp]
c2 = err_ref[0] / (h_ref[0] ** 2)
ax2.plot(h_ref, c2 * h_ref**2, 'k--', label=r'$O(h^2)$')

ax2.set_xlabel(r'Шаг по пространству $h$')
ax2.set_ylabel('Максимальная погрешность в конечный момент')
ax2.set_title('Сходимость по пространству (фиксированный $\tau$)')
ax2.grid(True, which="both", ls=":", linewidth=0.5)
ax2.set_xscale('log')
ax2.set_yscale('log')
ax2.legend()

plt.tight_layout()
plt.show()

def main_convergence():
    a = 1.0
    x_range = [0.0, np.pi]
    t_range = [0.0, 1.0]

    # Сходимость по времени (фиксированный h)
    N_fixed = 50
    h_fixed = (x_range[1] - x_range[0]) / N_fixed

    # Различные шаги по времени
    K_values = np.array([50, 100, 200, 400, 800])
    tau_values = (t_range[1] - t_range[0]) / K_values

    taus, err_tau_dict = convergence_in_time(a, h_fixed, t_range,
tau_values, x_range)

    # Сходимость по пространству (фиксированный tau)
    K_fixed = 500
    tau_fixed = (t_range[1] - t_range[0]) / K_fixed

    # Различные шаги по пространству
    N_values = np.array([20, 30, 40, 60, 80, 100])
    h_values = (x_range[1] - x_range[0]) / N_values

    hs, err_h_dict = convergence_in_space(a, tau_fixed, t_range,
h_values, x_range)

    plot_convergence(taus, err_tau_dict, hs, err_h_dict)

if __name__ == "__main__":
    main()
    main_approx()
    main_init_approx()
    main_convergence()

```

Результат работы программы:

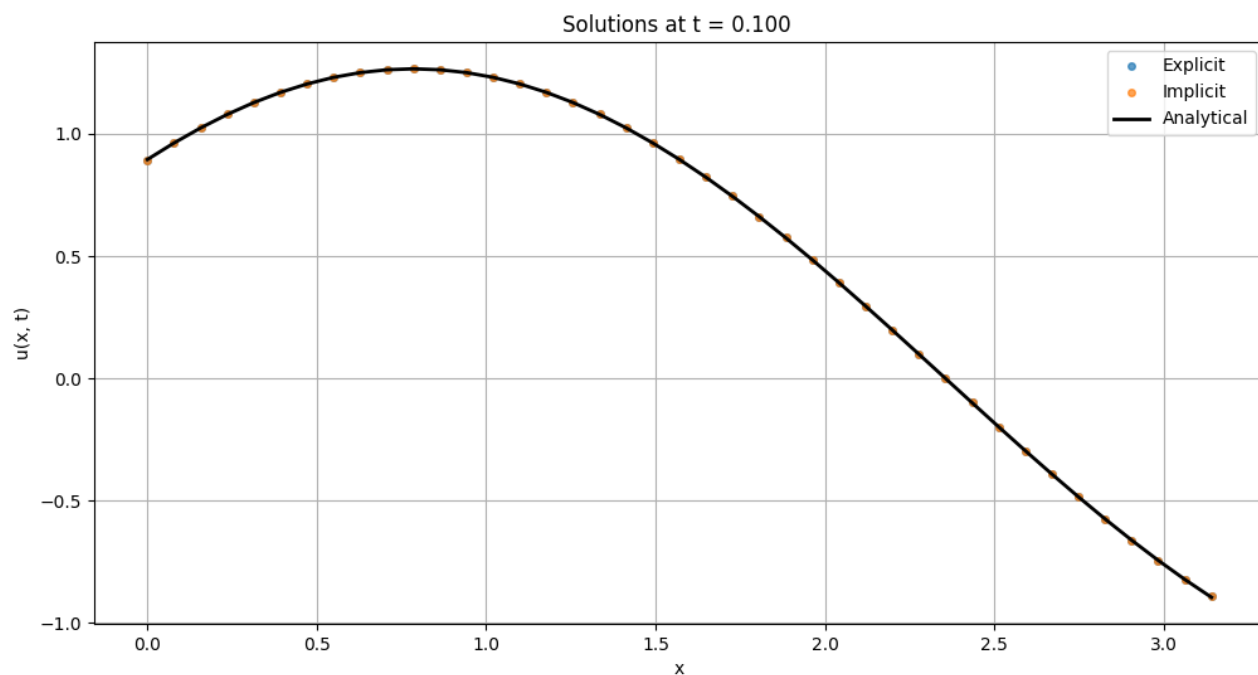


Рисунок 7. Сравнение численных решений с аналитическим в момент времени 0.1 секунда

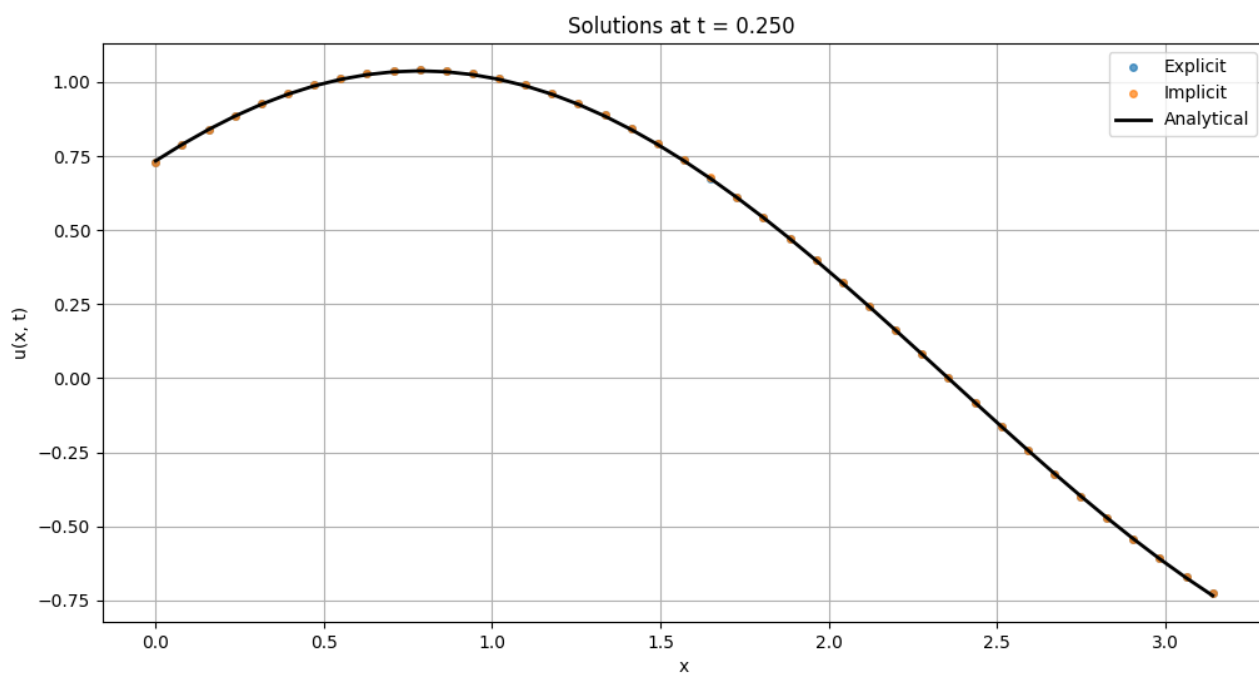


Рисунок 8. Сравнение численных решений с аналитическим в момент времени 0.25 секунд

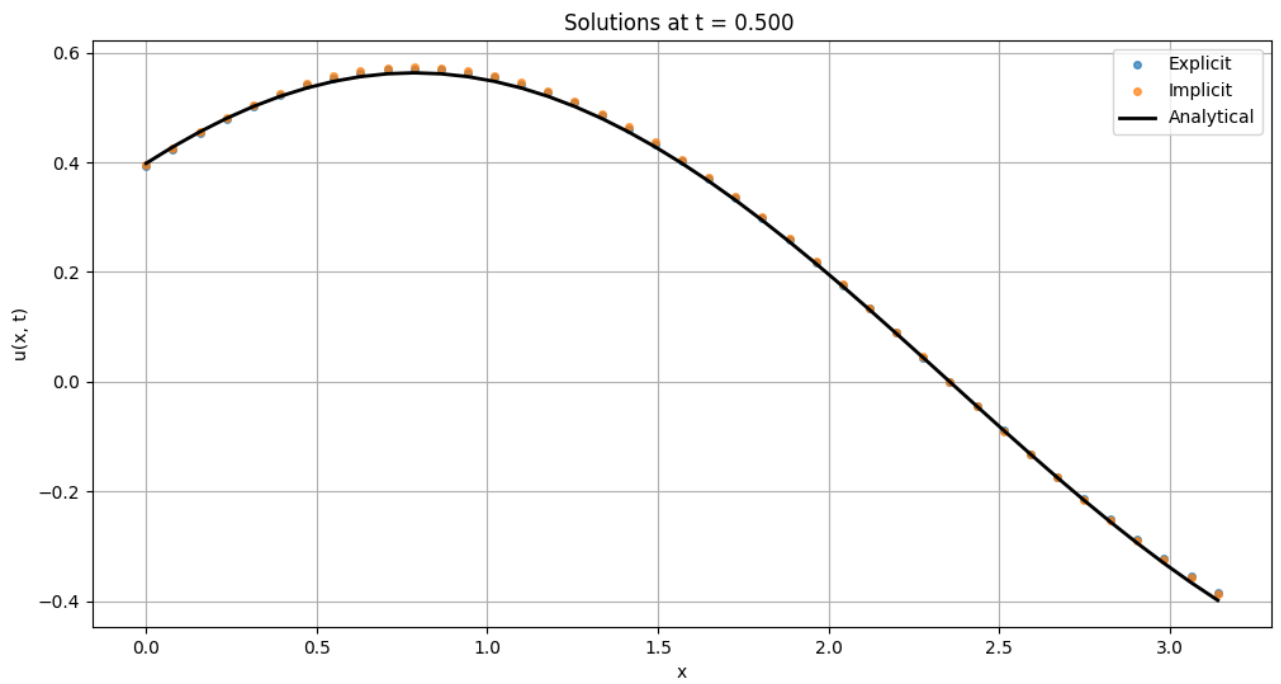


Рисунок 9. Сравнение численных решений с аналитическим в момент времени 0.5 секунд

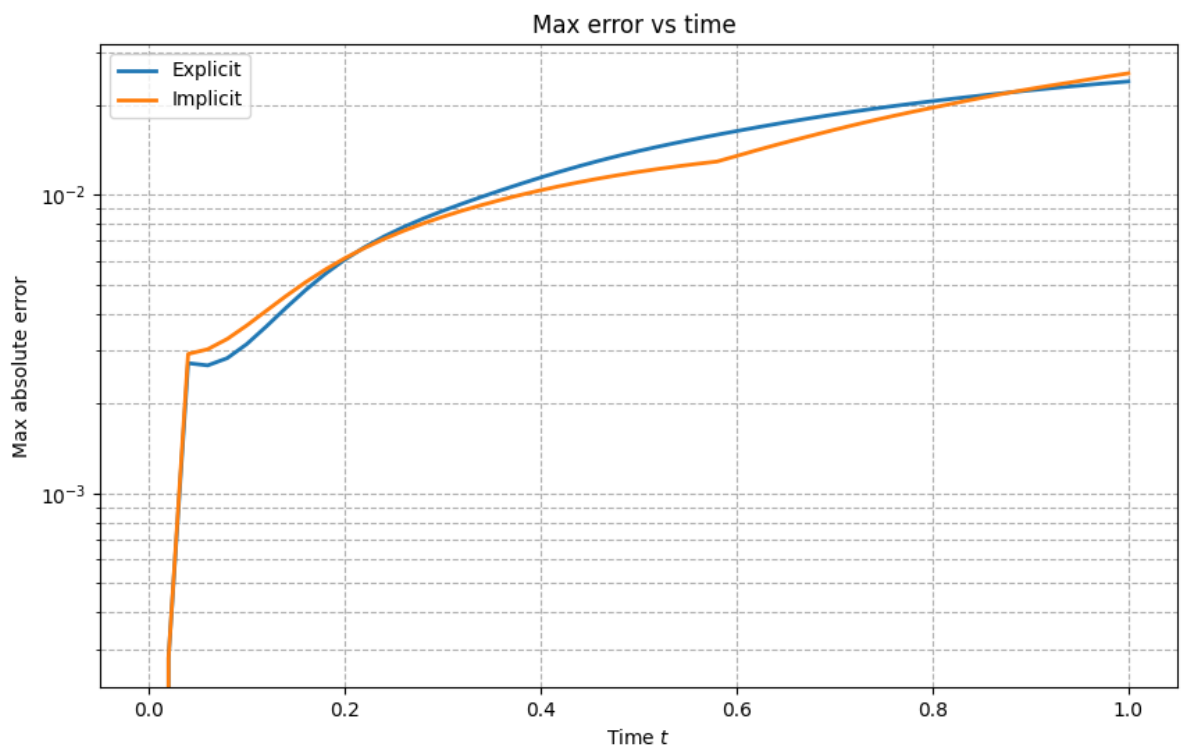


Рисунок 10. Зависимость погрешностей от времени

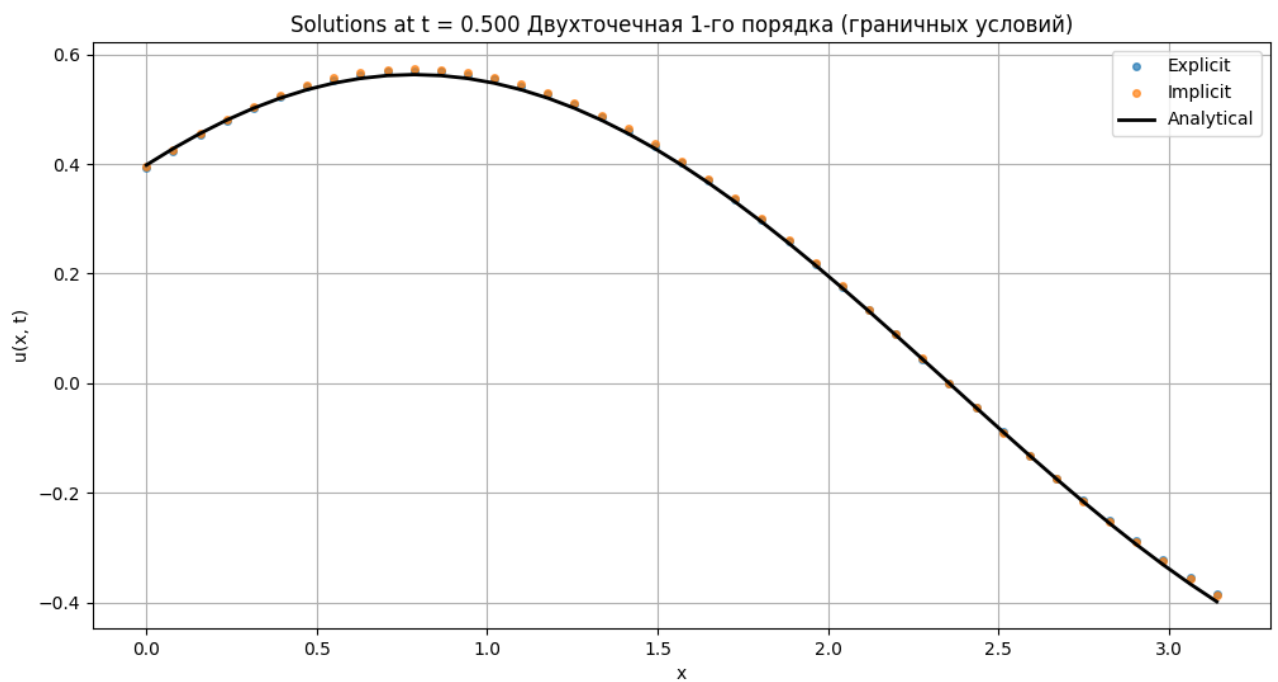


Рисунок 11. Сравнение численных решений с аналитическим в момент времени 0.5 секунд с двухточечной аппроксимацией 1-го порядка граничных условий

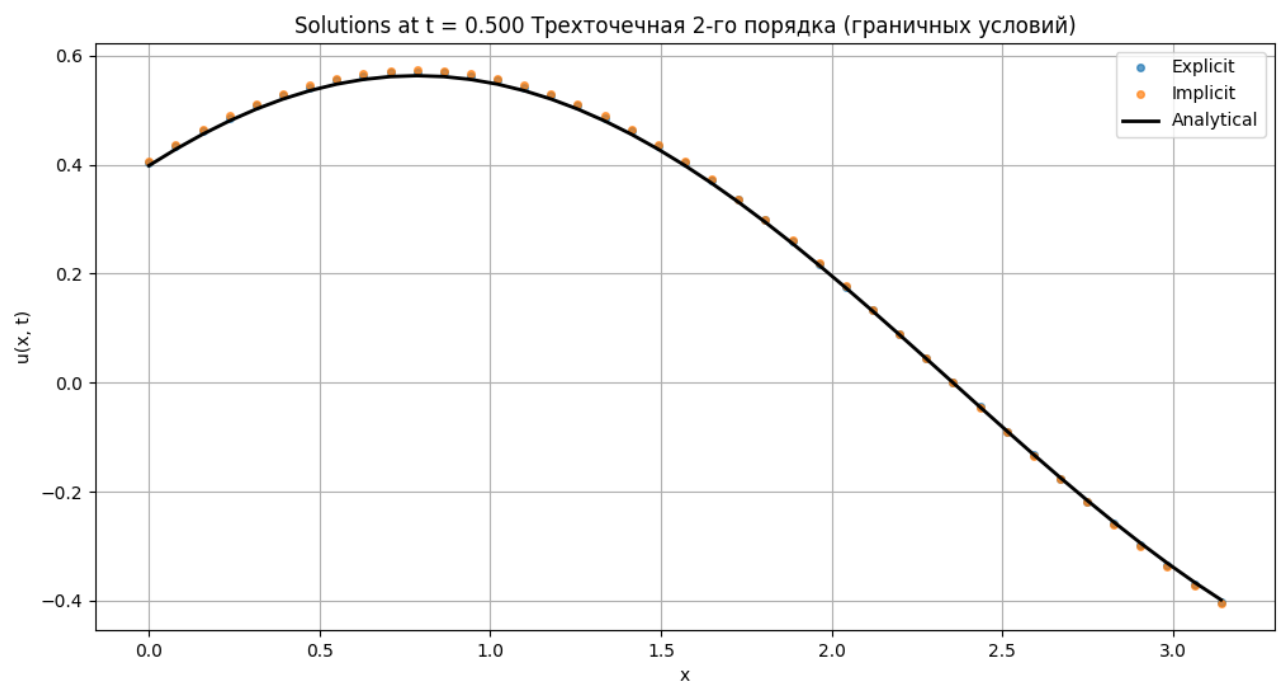


Рисунок 12. Сравнение численных решений с аналитическим в момент времени 0.5 секунд с трехточечной аппроксимацией 2-го порядка граничных условий

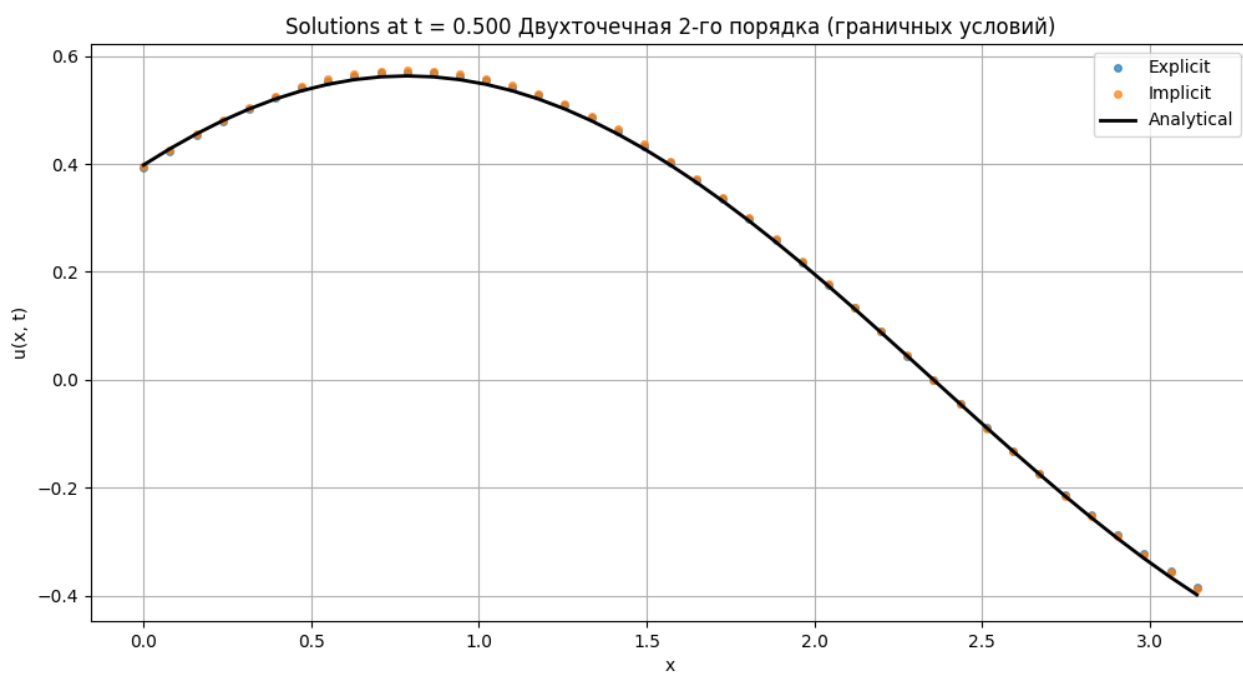


Рисунок 13. Сравнение численных решений с аналитическим в момент времени 0.5 секунд с двухточечной аппроксимацией 2-го порядка граничных условий

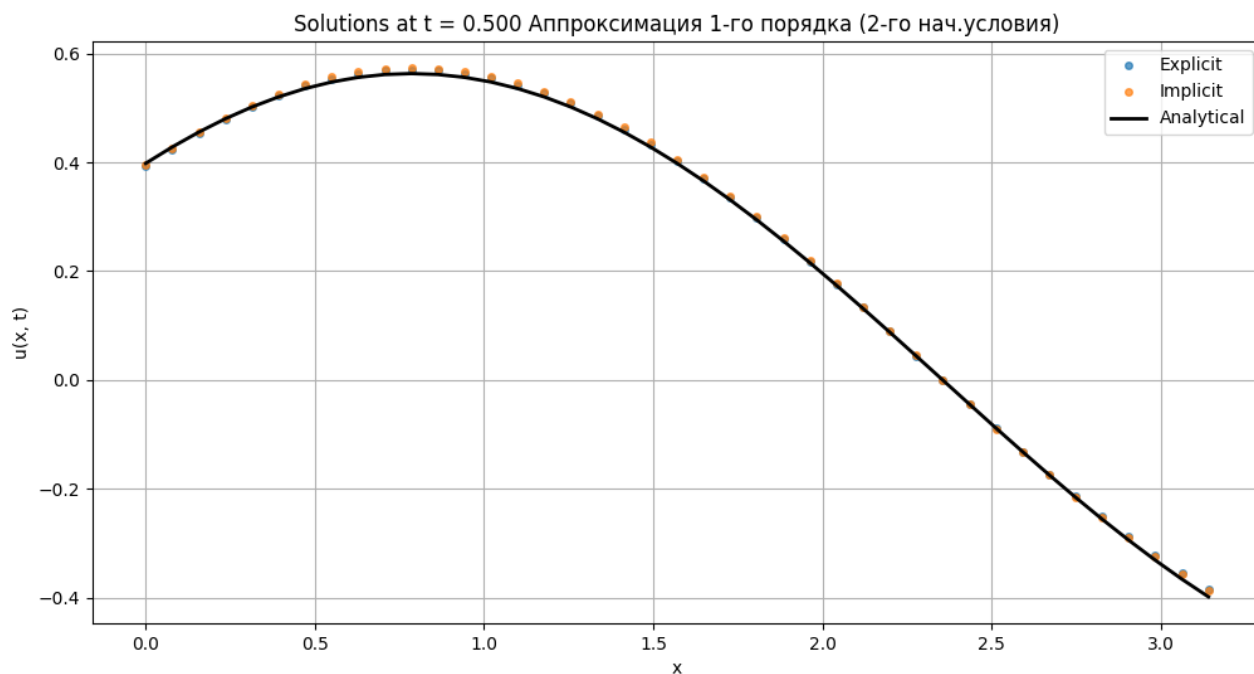


Рисунок 14. Сравнение численных решений с аналитическим в момент времени 0.5 секунд с аппроксимацией 1-го порядка 2-го начального условия

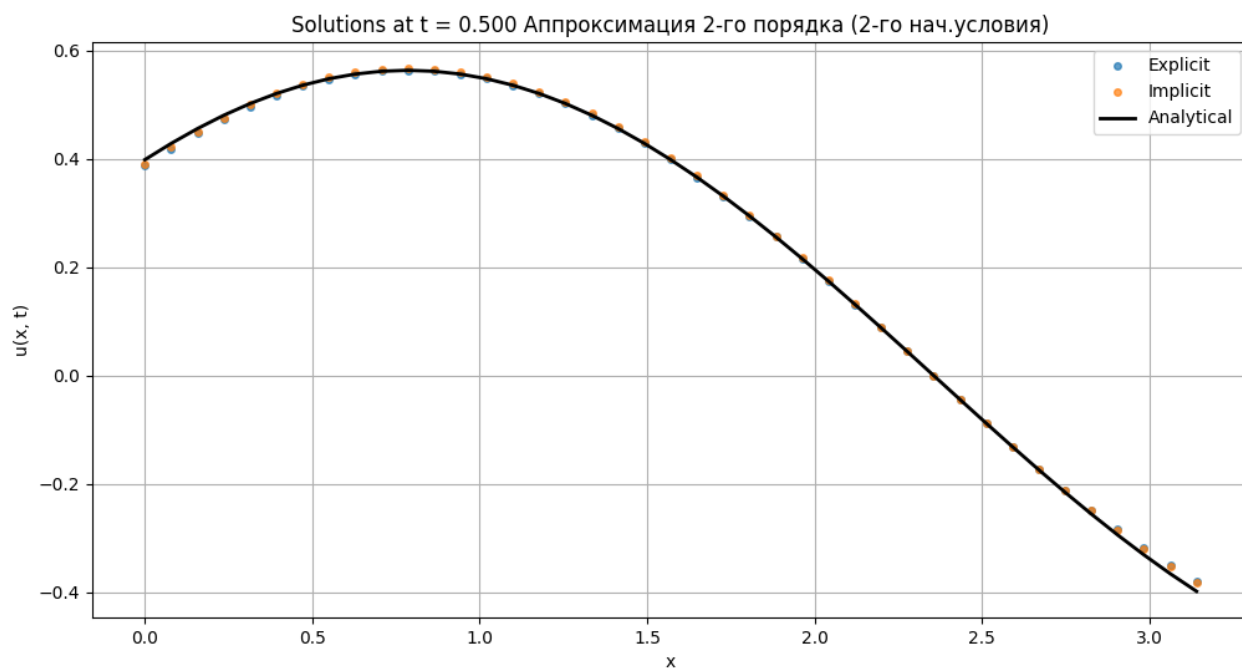


Рисунок 15. Сравнение численных решений с аналитическим в момент времени 0.5 секунд с аппроксимацией 2-го порядка 2-го начального условия

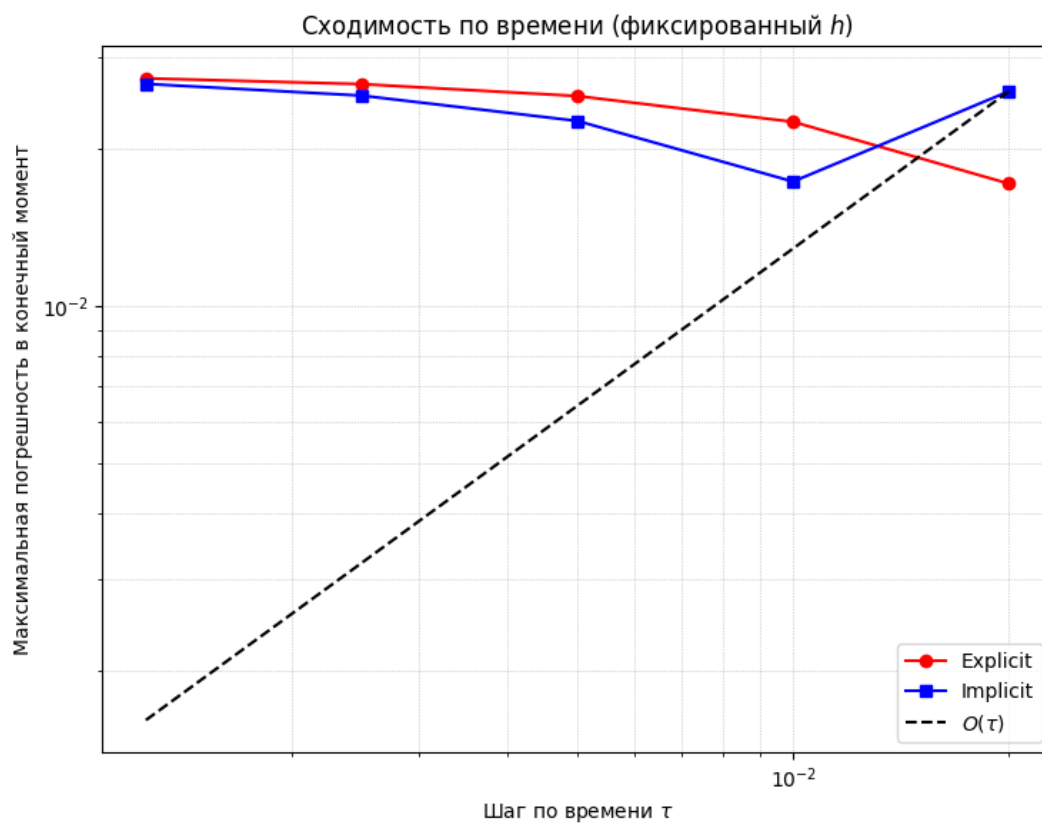


Рисунок 16. Зависимость погрешностей от шага по времени

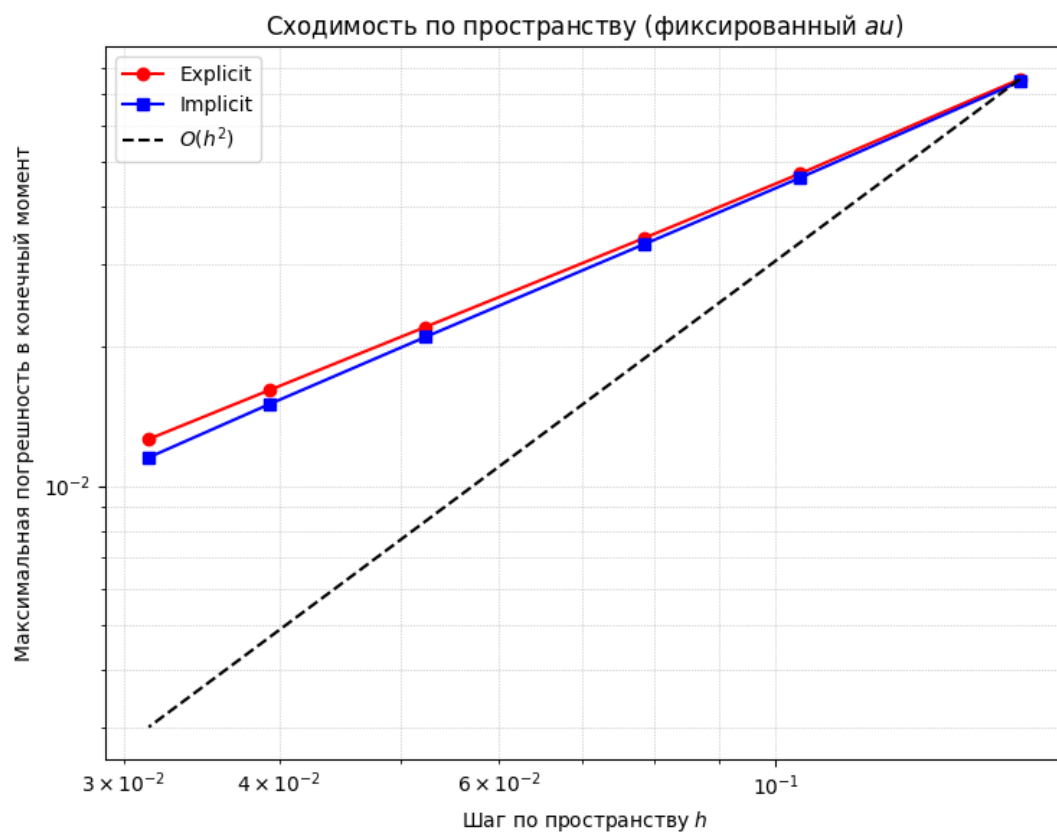


Рисунок 17. Зависимость погрешностей от шага по пространству

Лабораторная работа № 7 (3)

Задание: решить краевую задачу для дифференциального уравнения эллиптического типа. Аппроксимацию уравнения произвести с использованием центрально-разностной схемы. Для решения дискретного аналога применить следующие методы: метод простых итераций (метод Либмана), метод Зейделя, метод простых итераций с верхней релаксацией. Вычислить погрешность численного решения путем сравнения результатов с приведенным в задании аналитическим решением $U(x, t)$.

2.

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0,$$

$$u_x(0, y) = 0,$$

$$u(1, y) = 1 - y^2,$$

$$u_y(x, 0) = 0,$$

$$u(x, 1) = x^2 - 1.$$

Аналитическое решение: $U(x, y) = x^2 - y^2$

Описание решения:

Конечно-разностная схема уравнения имеет вид:

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_1^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_2^2}$$

В результате чего была получена пятидиагональная матрица СЛАУ. Для нахождения узлов применялись итерационные методы: метод Либмана (простых итераций), метод Зейделя и метод простых итераций с верхней релаксацией.

Метод Либмана: на каждой итерации значение в узле обновляется по формуле:

$$u_{i,j}^{k+1} = \left[\frac{1}{2 \left(\frac{1}{h_x^2} + \frac{1}{h_y^2} \right)} \right] \times \left[\frac{u_{i+1,j}^k + u_{i-1,j}^k}{h_x^2} + \frac{u_{i,j+1}^k + u_{i,j-1}^k}{h_y^2} \right]$$

Метод Зейделя: обновление значений производится с использованием уже вычисленных значений на текущей итерации:

$$u_{i,j}^{k+1} = \left[\frac{1}{2 \left(\frac{1}{h_x^2} + \frac{1}{h_y^2} \right)} \right] \times \left[\frac{u_{i+1,j}^{\{k\}} + u_{i-1,j}^{\{k+1\}}}{h_x^2} + \frac{u_{i,j+1}^{\{k\}} + u_{i,j-1}^{\{k+1\}}}{h_y^2} \right]$$

Метод простых итераций с верхней релаксацией: вводится параметр релаксации ω для ускорения сходимости:

$$u_{i,j}^{k+1} = (1 - \omega)u_{i,j}^k + \omega \times \left[\frac{1}{2 \left(\frac{1}{h_x^2} + \frac{1}{h_y^2} \right)} \right] \times \left[\frac{u_{i+1,j}^k + u_{i-1,j}^k}{h_x^2} + \frac{u_{i,j+1}^k + u_{i,j-1}^k}{h_y^2} \right]$$

Текст программы:

```
import matplotlib.pyplot as plt
import numpy as np
from copy import deepcopy
from matplotlib.colors import LinearSegmentedColormap

plt.rcParams['figure.figsize'] = [10, 10]

# Граничные условия
def phi1(y):
    return 0

def phi2(y):
    return 1 - y * y

def phi3(x):
    return 0

def phi4(x):
    return x * x - 1

# Аналитическое решение
def U(x, y):
    return x * x - y * y

# Норма
def norm(v1, v2):
    return np.amax(np.abs(v1 - v2))

# Функция для вычисления погрешностей
def error(lx, ly, Ny):
    Nx_array = [5, 10, 20, 40]
    size = np.size(Nx_array)
    hx_array = np.zeros(size)
    errors1 = np.zeros(size)
    errors2 = np.zeros(size)
    errors3 = np.zeros(size)
    for i in range(0, size):
```

```

hx_array[i] = lx / Nx_array[i]
x_array = np.arange(0, lx + hx_array[i], hx_array[i])
u1, tmp = Liebman(hx_array[i], hy, Nx_array[i], Ny)
u2, tmp = Seidel(hx_array[i], hy, Nx_array[i], Ny)
u3, tmp = UpperRelaxation(hx_array[i], hy, Nx_array[i], Ny)
if (np.size(x_array) != Nx_array[i] + 1):
    x_array = x_array[:Nx_array[i] + 1]
y = hy * Ny / 5
u_correct = np.zeros(np.size(x_array))
for j in range(np.size(x_array)):
    u_correct[j] = U(x_array[j], y)
u1_calculated = u1[:, int(Ny / 5)]
u2_calculated = u2[:, int(Ny / 5)]
u3_calculated = u3[:, int(Ny / 5)]
errors1[i] = norm(u_correct, u1_calculated)
errors2[i] = norm(u_correct, u2_calculated)
errors3[i] = norm(u_correct, u3_calculated)
return Nx_array, errors1, errors2, errors3

```

Функция для построения графика ошибок

```

def show_errors(lx, hy, Ny):
    Nx_array, errors1, errors2, errors3 = error(lx, hy, Ny)
    colors = ['blue', 'green', 'red']

    fig, ax = plt.subplots()
    plt.plot(Nx_array, errors1, color=colors[0], label='Метод
либмана', marker='o')
    plt.plot(Nx_array, errors2, color=colors[1], label='Метод
Зейделя', marker='s')
    plt.plot(Nx_array, errors3, color=colors[2], label='Метод простых
итераций с верхней релаксацией', marker='^')

    ax.set_xlabel('Количество узлов сетки Nx')
    ax.set_ylabel('Погрешность')
    ax.set_title('Зависимость погрешности от числа узлов сетки')
    ax.set_yscale('log') # Рекомендуется для лучшей визуализации
сходимости
    plt.grid(True, which="both", ls="--", linewidth=0.5)
    ax.legend()
    plt.show()

```

Функция для отрисовки решения в сечениях по y

```

def show_solution_slices(Nx, Ny, hx, hy, U, ulieb, usei, usor,
y_slices=[0.2, 0.5, 0.8]):
    # y_slices: [0.2, 0.5, 0.8] => y = 0.2*ly, 0.5*ly, 0.8*ly
    x_array = np.array([i * hx for i in range(Nx + 1)])
    colors = ['black', 'blue', 'green', 'red']

    for y_ratio in y_slices:
        y_fixed = y_ratio * ly
        y_index = int(y_fixed / hy)
        if y_index >= Ny + 1:
            y_index = Ny

        fig, ax = plt.subplots()
        u_correct = U(x_array, y_fixed)
        u_liebman = ulieb[:, y_index] # Сечение по y (столбец)
        u_seidel = usei[:, y_index]
        u_sor = usor[:, y_index]

        plt.plot(x_array, u_correct, color=colors[0], label=f'Точное
решение')
        plt.plot(x_array, u_liebman, color=colors[1], label='Метод
либмана')

```

```

plt.plot(x_array, u_seidel, color=colors[2], label='Метод
Зейделя')
plt.plot(x_array, u_sor, color=colors[3], label='Метод простых
итераций с верхней релаксацией')

ax.set_xlabel('x')
ax.set_ylabel('U(x, y)')
ax.set_title(f'Решение при y = {y_fixed:.2f}')
plt.grid()
ax.legend()
plt.show()

```

```

# Функция для построения трёхмерного графика решения
def show_solution3d(Nx, Ny, hx, hy, u, method_name, elev=45, azimuth=45):
    x = np.array([i * hx for i in range(Nx + 1)])
    y = np.array([j * hy for j in range(Ny + 1)])
    xgrid, ygrid = np.meshgrid(x, y)
    z = u
    fig = plt.figure()
    axes = fig.add_subplot(projection='3d')
    cmap = LinearSegmentedColormap.from_list('red_blue', ['b', 'r'],
256)
    axes.view_init(elev=elev, azimuth=azimuth)
    axes.set_xlabel('x')
    axes.set_ylabel('y')
    axes.set_zlabel('U(x,y)')
    axes.set_title('3D solution: ' + method_name)
    if z.shape != (len(y), len(x)):
        z = z.T
    axes.plot_surface(xgrid, ygrid, z, cmap=cmap, rcount=Ny + 1,
ccount=Nx + 1, edgecolor='none')
    plt.show()

```

```

# Метод простых итераций (Либмана)
def Liebman(hx, hy, Nx, Ny, eps = 1e-5):
    u = np.zeros((Nx + 1, Ny + 1))
    # Граничные условия 1-го рода
    for i in range(Nx + 1):
        u[i][Ny] = phi4(i * hx)
    for j in range(Ny + 1):
        u[Nx][j] = phi2(j * hy)

    iter_num = 0
    while True:
        u_next = deepcopy(u)
        for i in range(1, Nx):
            for j in range(1, Ny):
                u_next[i][j] = 1 / (2 / hx ** 2 + 2 / hy ** 2) * ((u[i
- 1][j] + u[i + 1][j]) / hx ** 2 + (u[i][j - 1] + u[i][j + 1]) / hy **
2)
        # Граничные условия 2-го рода
        for i in range(Nx):
            u_next[i][0] = u_next[i][1] - hy * phi3(i * hx)
        for j in range(Ny):
            u_next[0][j] = u_next[1][j] - hx * phi1(j * hy)

        if norm(u, u_next) < eps:
            break
        u = deepcopy(u_next)
        iter_num += 1
    return u, iter_num

```

```

# Метод Зейделя

```



```

def seidel(hx, hy, Nx, Ny, eps = 1e-5):
    u = np.zeros((Nx + 1, Ny + 1))
    # Граничные условия 1-го рода
    for i in range(Nx + 1):
        u[i][Ny] = phi4(i * hx)
    for j in range(Ny + 1):
        u[Nx][j] = phi2(j * hy)

    iter_num = 0
    while True:
        u_prev = deepcopy(u)
        u_next = deepcopy(u)
        for i in range(1, Nx):
            for j in range(1, Ny):
                u_next[i][j] = 1 / (2 / hx ** 2 + 2 / hy ** 2) *
                ((u_next[i - 1][j] + u_prev[i + 1][j]) / hx ** 2 + (u_next[i][j - 1] +
                u_prev[i][j + 1]) / hy ** 2)
        # Граничные условия 2-го рода
        for i in range(Nx):
            u_next[i][0] = u_next[i][1] - hy * phi3(i * hx)
        for j in range(Ny):
            u_next[0][j] = u_next[1][j] - hx * phi1(j * hy)

        if norm(u_prev, u_next) < eps:
            break
        u = deepcopy(u_next)
        iter_num += 1
    return u, iter_num

```

```

# Метод простых итераций с верхней релаксации
def upperRelaxation(hx, hy, Nx, Ny, eps = 1e-5, omega = 1.25):
    u = np.zeros((Nx + 1, Ny + 1))
    # Граничные условия 1-го рода
    for i in range(Nx + 1):
        u[i][Ny] = phi4(i * hx)
    for j in range(Ny + 1):
        u[Nx][j] = phi2(j * hy)

    iter_num = 0
    while True:
        u_prev = deepcopy(u)
        u_next = deepcopy(u)
        for i in range(1, Nx):
            for j in range(1, Ny):
                u_next[i][j] = 1 / (2 / hx ** 2 + 2 / hy ** 2) *
                ((u_next[i - 1][j] + u_prev[i + 1][j]) / hx ** 2 + (u_next[i][j - 1] +
                u_prev[i][j + 1]) / hy ** 2)
        # Граничные условия 2-го рода
        for i in range(Nx):
            u_next[i][0] = u_next[i][1] - hy * phi3(i * hx)
        for j in range(Ny):
            u_next[0][j] = u_next[1][j] - hx * phi1(j * hy)
        # Релаксация
        u_next = omega * u_next + (1 - omega) * u_prev

        if norm(u_prev, u_next) < eps:
            break
        u = deepcopy(u_next)
        iter_num += 1
    return u, iter_num

```

```

def main():
    global lx, ly
    lx = ly = 1

```

```

Nx = Ny = 30
hx = lx / Nx
hy = ly / Ny

u1, iter_num1 = Liebman(hx, hy, Nx, Ny)
print("Кол-во итераций метода Либмана:", iter_num1)
u2, iter_num2 = Seidel(hx, hy, Nx, Ny)
print("Кол-во итераций метода Зейделя:", iter_num2)
u3, iter_num3 = UpperRelaxation(hx, hy, Nx, Ny)
print("Кол-во итераций метода простых итераций с верхней
релаксацией:", iter_num3)

# Построение 3D графика
show_solution3d(Nx, Ny, hx, hy, u1, "Liebman_method", elev=20,
azim=110)
show_solution3d(Nx, Ny, hx, hy, u2, "Seidel_method", elev=20,
azim=110)
show_solution3d(Nx, Ny, hx, hy, u3, "UpperRelaxation_method",
elev=20, azim=110)

# Построение графиков решений в сечениях по y
show_solution_slices(Nx, Ny, hx, hy, u1, u2, u3, y_slices=[0.2,
0.5, 0.8])

# График погрешностей
show_errors(lx, hy, Ny)

# Функция для вычисления погрешностей в зависимости от hx (при
фиксированном hy)
def error_hx_dependence(lx, hy, Ny):
    Nx_array = [5, 10, 20, 40]
    size = np.size(Nx_array)
    hx_array = np.zeros(size)
    errors1 = np.zeros(size)
    errors2 = np.zeros(size)
    errors3 = np.zeros(size)
    for i in range(0, size):
        hx_array[i] = lx / Nx_array[i]
        x_array = np.arange(0, lx + hx_array[i], hx_array[i])
        u1, tmp = Liebman(hx_array[i], hy, Nx_array[i], Ny)
        u2, tmp = Seidel(hx_array[i], hy, Nx_array[i], Ny)
        u3, tmp = UpperRelaxation(hx_array[i], hy, Nx_array[i], Ny)

        if len(x_array) > Nx_array[i] + 1:
            x_array = x_array[:Nx_array[i] + 1]
        elif len(x_array) < Nx_array[i] + 1:
            x_array = np.append(x_array, lx)

        # Выберем сечение по y = 0.2 * ly
        y_fixed = 0.2 * ly
        y_index = int(y_fixed / hy) # индекс строки в сетке,
соответствующий y_fixed
        if y_index >= Ny + 1:
            y_index = Ny # Защита от выхода за границы

        u_correct = np.zeros(len(x_array))
        for j in range(len(x_array)):
            u_correct[j] = U(x_array[j], y_fixed)

        u1_calculated = u1[:, y_index]
        u2_calculated = u2[:, y_index]
        u3_calculated = u3[:, y_index]

        errors1[i] = norm(u_correct, u1_calculated)
        errors2[i] = norm(u_correct, u2_calculated)

```

```

        errors3[i] = norm(u_correct, u3_calculated)

    return hx_array, errors1, errors2, errors3

# функция для вычисления погрешностей в зависимости от hy (при
# фиксированном hx)
def error_hy_dependence(lx, hx, Nx):
    Ny_array = [5, 10, 20, 40]
    size = np.size(Ny_array)
    hy_array = np.zeros(size)
    errors1 = np.zeros(size)
    errors2 = np.zeros(size)
    errors3 = np.zeros(size)
    for i in range(0, size):
        hy_array[i] = ly / Ny_array[i]
        y_array = np.arange(0, ly + hy_array[i], hy_array[i])
        u1, tmp = Liebman(hx, hy_array[i], Nx, Ny_array[i])
        u2, tmp = Seidel(hx, hy_array[i], Nx, Ny_array[i])
        u3, tmp = UpperRelaxation(hx, hy_array[i], Nx, Ny_array[i])
        # убедимся, что размеры совпадают
        if len(y_array) > Ny_array[i] + 1:
            y_array = y_array[:Ny_array[i] + 1]
        elif len(y_array) < Ny_array[i] + 1:
            y_array = np.append(y_array, ly)

        # выберем сечение по x = 0.3 * lx
        x_fixed = 0.3 * lx
        x_index = int(x_fixed / hx) # индекс столбца в сетке,
соответствующий x_fixed
        if x_index >= Nx + 1:
            x_index = Nx

        u_correct = np.zeros(len(y_array))
        for j in range(len(y_array)):
            u_correct[j] = U(x_fixed, y_array[j])

        u1_calculated = u1[x_index, :]
        u2_calculated = u2[x_index, :]
        u3_calculated = u3[x_index, :]

        errors1[i] = norm(u_correct, u1_calculated)
        errors2[i] = norm(u_correct, u2_calculated)
        errors3[i] = norm(u_correct, u3_calculated)

    return hy_array, errors1, errors2, errors3

# функция для построения графика ошибок в зависимости от hx
def show_errors_hx(lx, hy, Ny):
    hx_array, errors1, errors2, errors3 = error_hx_dependence(lx, hy,
Ny)
    colors = ['blue', 'green', 'red']
    fig, ax = plt.subplots()
    plt.plot(hx_array, errors1, color=colors[0], label='Метод
либмана')
    plt.plot(hx_array, errors2, color=colors[1], label='Метод
Зейделя')
    plt.plot(hx_array, errors3, color=colors[2], label='Метод простых
итераций с верхней релаксацией')
    ax.set_xlabel('h_x')
    ax.set_ylabel('Погрешность')
    ax.set_title('Зависимость погрешности от шага h_x')
    plt.grid()
    ax.legend()
    plt.show()

```

```

# функция для построения графика ошибок в зависимости от hy
def show_errors_hy(lx, hx, Nx):
    hy_array, errors1, errors2, errors3 = error_hy_dependence(lx, hx,
Nx)
    colors = ['blue', 'green', 'red']
    fig, ax = plt.subplots()
    plt.plot(hy_array, errors1, color=colors[0], label='Метод
либмана')
    plt.plot(hy_array, errors2, color=colors[1], label='Метод
зейделя')
    plt.plot(hy_array, errors3, color=colors[2], label='Метод простых
итераций с верхней релаксацией')
    ax.set_xlabel('h_y')
    ax.set_ylabel('Погрешность')
    ax.set_title('Зависимость погрешности от шага h_y')
    plt.grid()
    ax.legend()
    plt.show()

def main_convergence():
    Nx = Ny = 30
    hx = lx / Nx
    hy = ly / Ny

    # Исследование зависимости погрешности от hx
    show_errors_hx(lx, hy, Ny)

    # Исследование зависимости погрешности от hy
    show_errors_hy(lx, hx, Nx)

if __name__ == "__main__":
    main()
    # main_convergence()

```

Результат работы программы:

кол-во итераций метода либмана: 851

кол-во итераций метода Зейделя: 498

кол-во итераций метода простых итераций с верхней релаксацией: 411

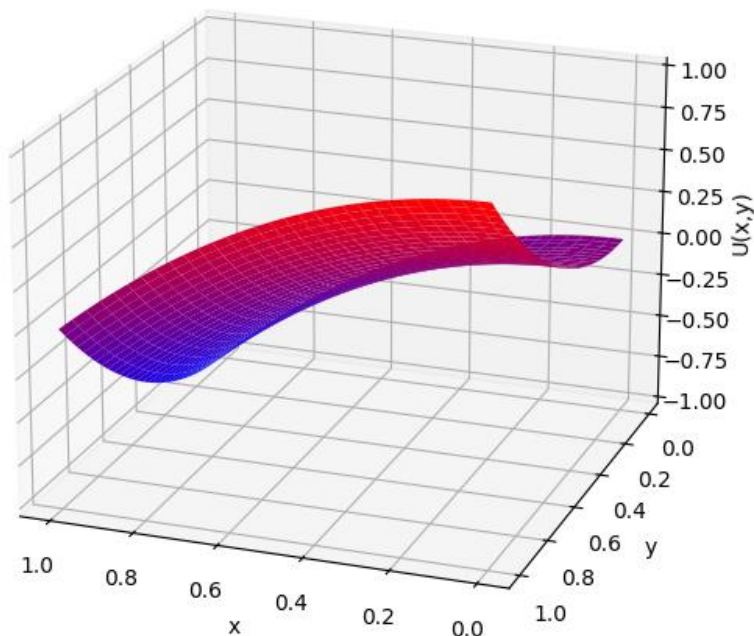


Рисунок 18. Трехмерная визуализация метода Либмана

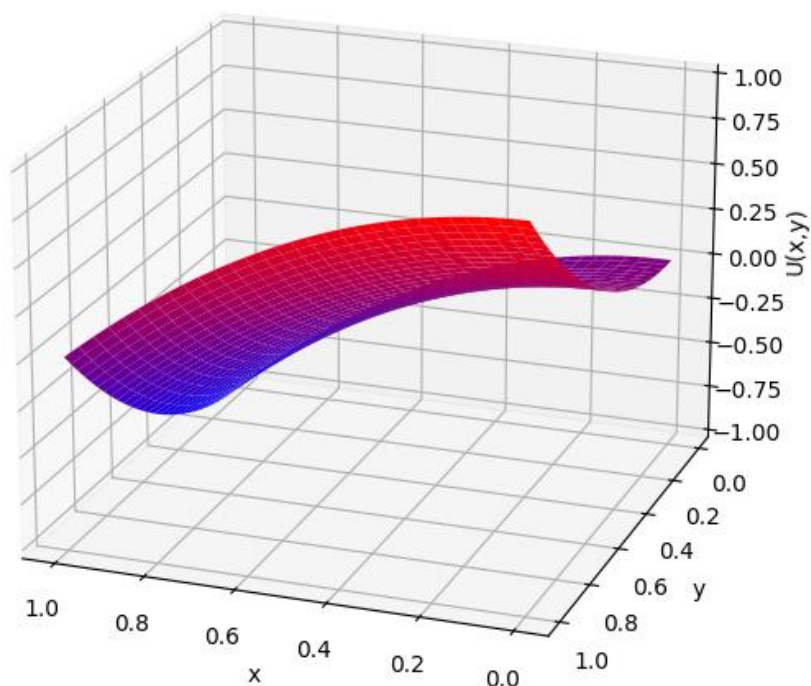


Рисунок 19. Трехмерная визуализация метода Зейделя

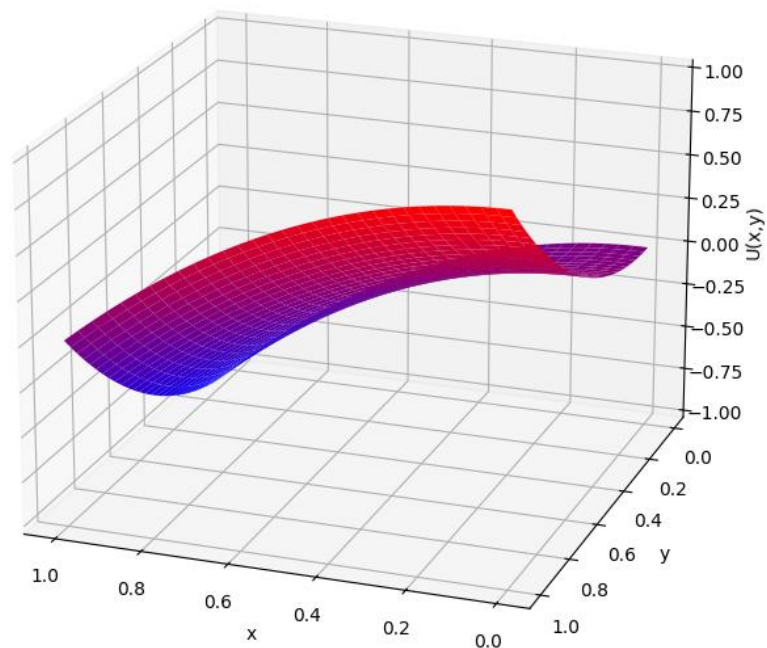


Рисунок 20. Трехмерная визуализация метода простых итерация с верхней релаксацией

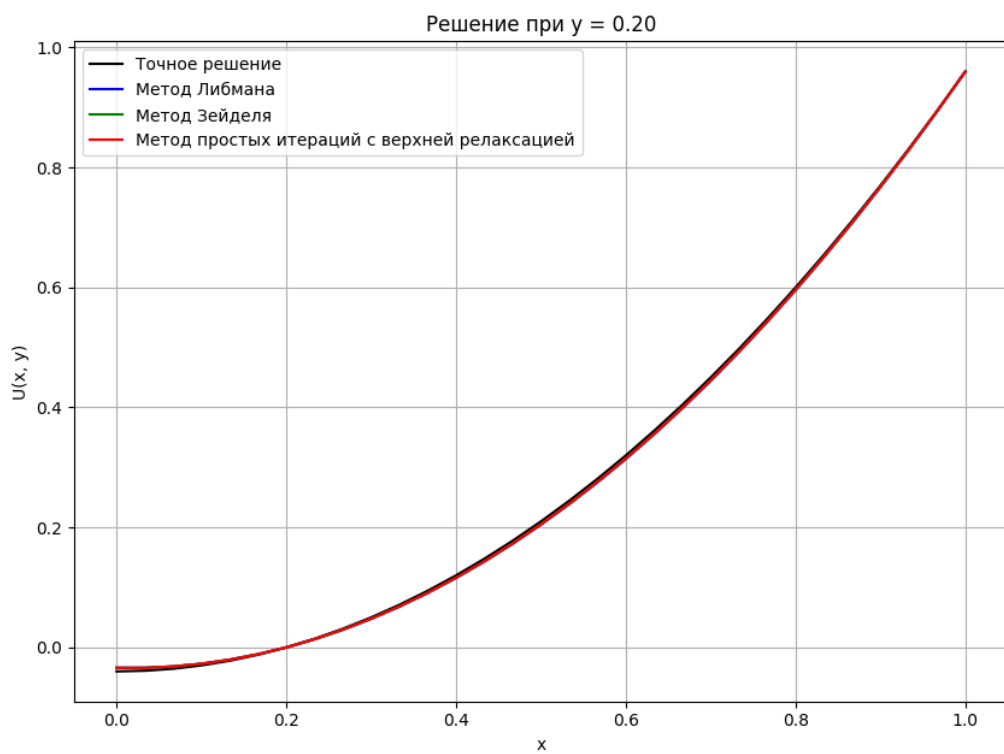


Рисунок 21. Сравнение численных решений с аналитическим при $y = 0.20$

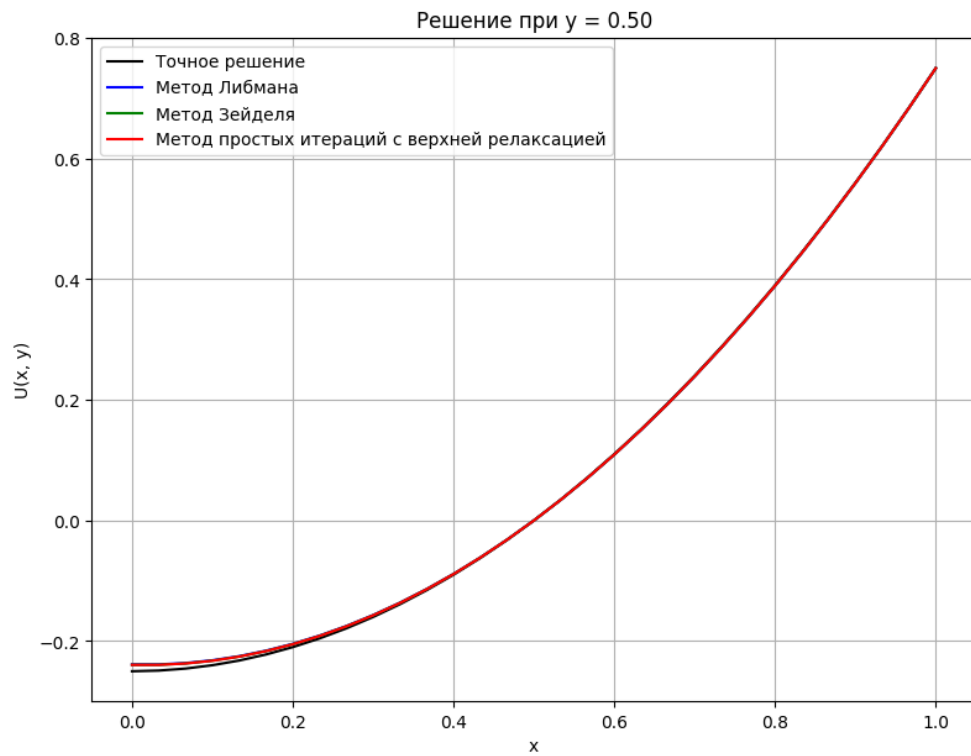


Рисунок 22. Сравнение численных решений с аналитическим при $y = 0.50$

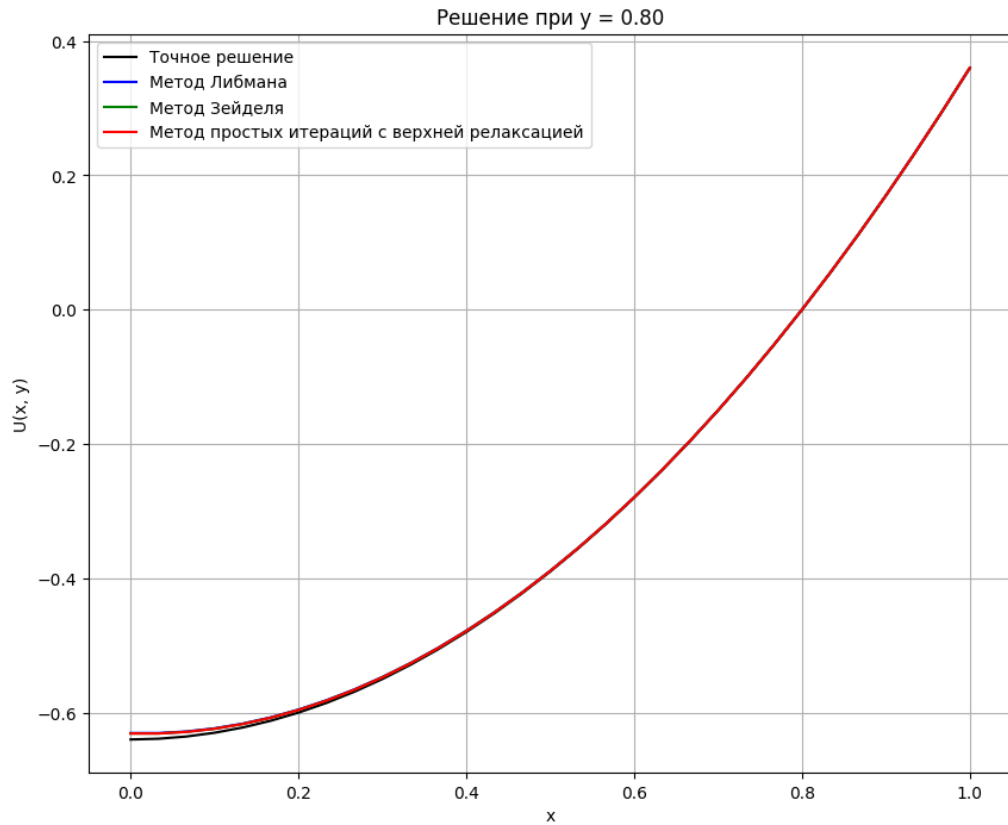


Рисунок 23. Сравнение численных решений с аналитическим при $y = 0.80$

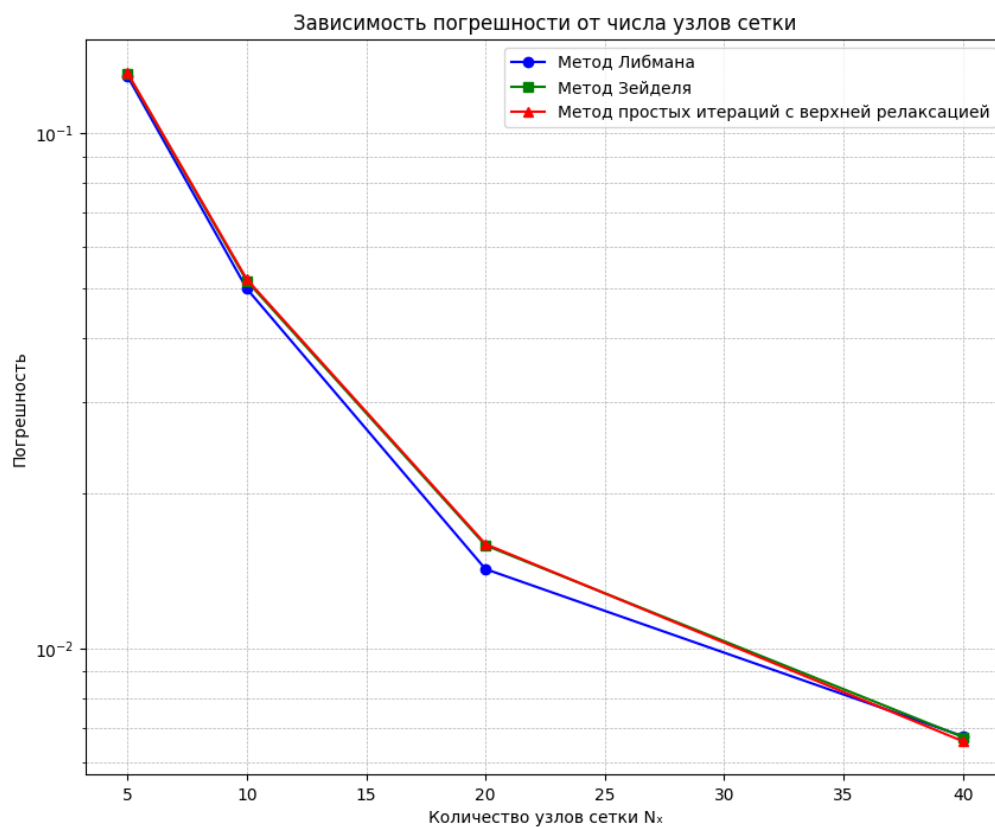


Рисунок 24. Зависимость погрешностей от количества узлов сетки

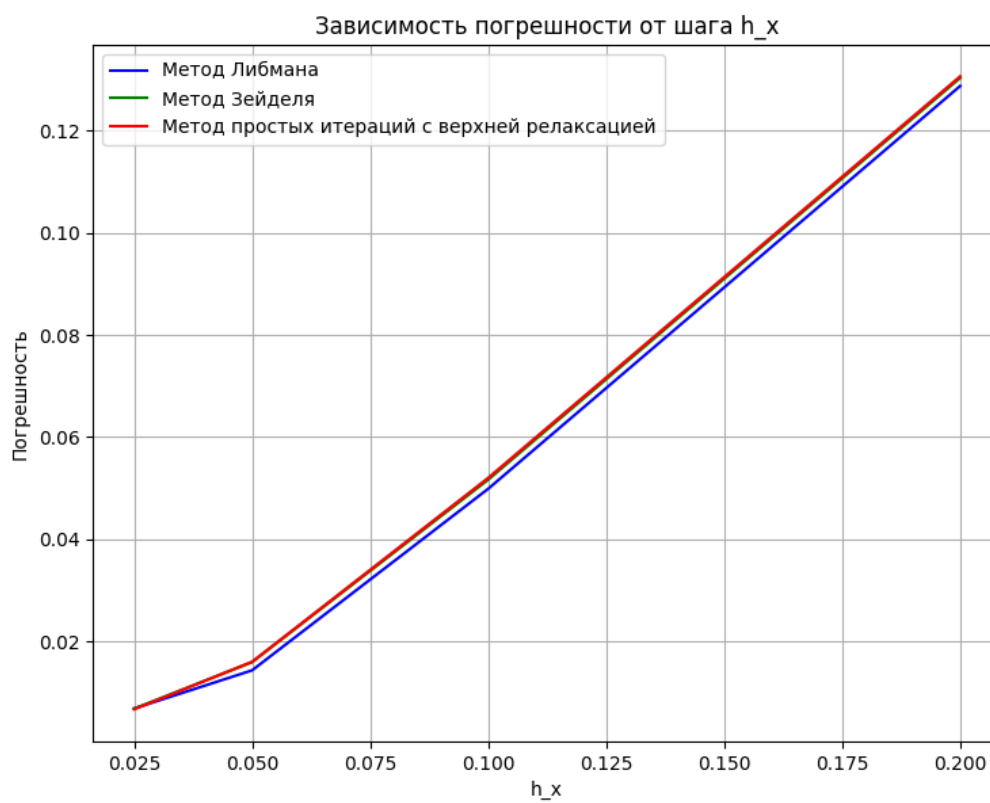


Рисунок 25. Зависимость погрешностей от шага h_x

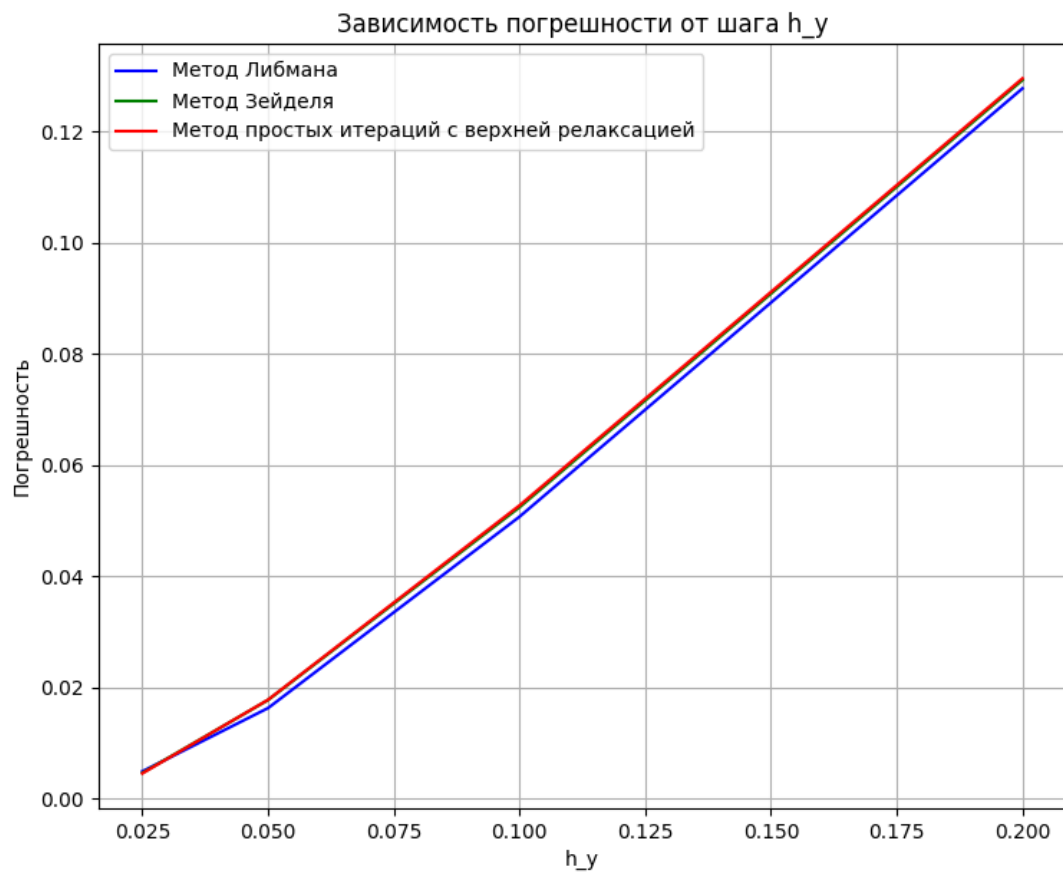


Рисунок 26. Зависимость погрешностей от шага h_y

Лабораторная работа № 8 (4)

Задание: используя схемы переменных направлений и дробных шагов, решить двумерную начально-краевую задачу для дифференциального уравнения параболического типа. В различные моменты времени вычислить погрешность численного решения путем сравнения результатов с приведенным в задании аналитическим решением $U(x, t)$.

2.

$$\frac{\partial u}{\partial t} = a \frac{\partial^2 u}{\partial x^2} + a \frac{\partial^2 u}{\partial y^2}, \quad a > 0,$$

$$u(0, y, t) = \cos(\mu_2 y) \exp(-(\mu_1^2 + \mu_2^2)at),$$

$$u\left(\frac{\pi}{2} \mu_1, y, t\right) = 0,$$

$$u(x, 0, t) = \cos(\mu_1 x) \exp(-(\mu_1^2 + \mu_2^2)at),$$

$$u\left(x, \frac{\pi}{2} \mu_2, t\right) = 0,$$

$$u(x, y, 0) = \cos(\mu_1 x) \cos(\mu_2 y).$$

$$\text{Аналитическое решение: } U(x, y, t) = \cos(\mu_1 x) \cos(\mu_2 y) \exp(-(\mu_1^2 + \mu_2^2)at).$$

1). $\mu_1 = 1, \mu_2 = 1.$

2). $\mu_1 = 2, \mu_2 = 1.$

3). $\mu_1 = 1, \mu_2 = 2.$

Описание решения:

Рассматривается двумерное уравнение параболического типа – уравнение теплопроводности в прямоугольной области с начальным условием и граничными условиями первого рода.

В граничных условиях используются значения ($x = \frac{\pi}{2} \cdot \mu_1$ и $y = \frac{\pi}{2} \cdot \mu_2$), которые при подстановке подстановке в аналитическое решение не при всех вариантах указанных значений μ_1 и μ_2 дают верное равенство. Поэтому граничные значения определяются из аналитического решения.

Также вводится пространственно-временная сетка.

Для численного решения используются следующие методы: метод переменных направлений (МПН) и метод дробных шагов (МДШ).

Метод переменных направлений. Схема:

Первый дробный шаг ($t^k \rightarrow t^{k+\frac{1}{2}}$):

$$\frac{u_{i,j}^{k+\frac{1}{2}} - u_{i,j}^k}{\frac{\tau}{2}} = \frac{a \left(u_{i+1,j}^{k+\frac{1}{2}} - 2u_{i,j}^{k+\frac{1}{2}} + u_{i-1,j}^{k+\frac{1}{2}} \right)}{h_x^2} + \frac{a(u_{i,j+1}^k - 2u_{i,j}^k + u_{i,j-1}^k)}{h_y^2},$$

где:

Оператор по x: $\frac{\partial^2 u}{\partial x^2}$ - аппроксимируется неявно на слое $t^{k+\frac{1}{2}}$

Оператор по y: $\frac{\partial^2 u}{\partial y^2}$ - аппроксимируется явно на слое t^k

Второй дробный шаг ($t^{k+\frac{1}{2}} \rightarrow t^{k+1}$):

$$\frac{u_{ij}^{k+1} - u_{ij}^{k+\frac{1}{2}}}{\frac{\tau}{2}} = \frac{a \left(u_{i+1,j}^{k+\frac{1}{2}} - 2u_{ij}^{k+\frac{1}{2}} + u_{i-1,j}^{k+\frac{1}{2}} \right)}{h_x^2} + \frac{a(u_{i,j+1}^{k+1} - 2u_{ij}^{k+1} + u_{i,j-1}^{k+1})}{h_y^2},$$

где:

Оператор по x: $\frac{\partial^2 u}{\partial x^2}$ - аппроксимируется явно на слое $t^{k+\frac{1}{2}}$

Оператор по y: $\frac{\partial^2 u}{\partial y^2}$ - аппроксимируется неявно на слое t^{k+1}

Метод дробных шагов. Схема:

Первый шаг ($t^k \rightarrow t^{k+\frac{1}{2}}$):

$$\frac{u_{ij}^{k+\frac{1}{2}} - u_{ij}^k}{\tau} = \frac{a \left(u_{i+1,j}^{k+\frac{1}{2}} - 2u_{ij}^{k+\frac{1}{2}} + u_{i-1,j}^{k+\frac{1}{2}} \right)}{h_x^2},$$

где:

Оператор по x: $\frac{\partial^2 u}{\partial x^2}$ - аппроксимируется неявно на слое $t^{k+\frac{1}{2}}$

Оператор по y: $\frac{\partial^2 u}{\partial y^2}$ - отсутствует (расщепление операторов)

Второй шаг ($t^{k+\frac{1}{2}} \rightarrow t^{k+1}$):

$$\frac{u_{ij}^{k+1} - u_{ij}^{k+\frac{1}{2}}}{\tau} = \frac{a(u_{i,j+1}^{k+1} - 2u_{ij}^{k+1} + u_{i,j-1}^{k+1})}{h_y^2},$$

где:

Оператор по x: $\frac{\partial^2 u}{\partial x^2}$ - отсутствует (расщепление операторов)

Оператор по y: $\frac{\partial^2 u}{\partial y^2}$ - аппроксимируется неявно на слое t^{k+1}

Текст программы:

```
import matplotlib.pyplot as plt
import numpy as np
from copy import deepcopy

plt.rcParams['figure.figsize'] = [8, 7]
a = 1

def phi1(y, t, mu1, mu2): # x = 0
    return np.cos(mu2 * y) * np.exp(-(mu1**2 + mu2**2) * a * t)

def phi2(y, t, mu1, mu2): # x = Lx
    Lx = mu1 * np.pi / 2
    return np.cos(mu1 * Lx) * np.cos(mu2 * y) * np.exp(-(mu1**2 + mu2**2) * a * t)

def phi3(x, t, mu1, mu2): # y = 0
    return np.cos(mu1 * x) * np.exp(-(mu1**2 + mu2**2) * a * t)

def phi4(x, t, mu1, mu2): # y = Ly
    Ly = mu2 * np.pi / 2
    return np.cos(mu1 * x) * np.cos(mu2 * Ly) * np.exp(-(mu1**2 + mu2**2) * a * t)

def psi(x, y, mu1, mu2):
    return np.cos(mu1 * x) * np.cos(mu2 * y)

def U(x, y, t, mu1, mu2):
    return np.cos(mu1 * x) * np.cos(mu2 * y) * np.exp(-(mu1**2 + mu2**2) * a * t)

def Check(A):
    if A.shape[0] != A.shape[1]:
        return False
    n = A.shape[0]
    for i in range(n):
        diag = abs(A[i, i])
        off_diag = sum(abs(A[i, j]) for j in range(n) if j != i)
        if diag < off_diag:
            return False
    return True

def solve(A, b):
    if Check(A):
        n = len(b)
        p = np.zeros(n)
        q = np.zeros(n)
        p[0] = -A[0, 1] / A[0, 0]
        q[0] = b[0] / A[0, 0]
        for i in range(1, n - 1):
            denom = A[i, i] + A[i, i - 1] * p[i - 1]
            p[i] = -A[i, i + 1] / denom
            q[i] = (b[i] - A[i, i - 1] * q[i - 1]) / denom
        denom = A[n - 1, n - 1] + A[n - 1, n - 2] * p[n - 2]
        q[n - 1] = (b[n - 1] - A[n - 1, n - 2] * q[n - 2]) / denom
        x = np.zeros(n)
        x[-1] = q[-1]
        for i in range(n - 2, -1, -1):
            x[i] = p[i] * x[i + 1] + q[i]
        return x
    else:
        return np.linalg.solve(A, b)
```

```

def VariableDirectionMethod(Nt, Nx, Ny, tau, hx, hy, mu1, mu2):
    Lx = mu1 * np.pi / 2
    Ly = mu2 * np.pi / 2
    u = np.zeros((Nt + 1, Nx + 1, Ny + 1))
    x_grid = np.linspace(0, Lx, Nx + 1)
    y_grid = np.linspace(0, Ly, Ny + 1)

    # Начальное условие
    for i in range(Nx + 1):
        for j in range(Ny + 1):
            u[0, i, j] = psi(x_grid[i], y_grid[j], mu1, mu2) #
ИСПРАВЛЕНО: порядок аргументов

    # Граничные условия
    for k in range(Nt + 1):
        t = k * tau
        u[k, :, 0] = phi3(x_grid, t, mu1, mu2) # y = 0
        u[k, :, -1] = phi4(x_grid, t, mu1, mu2) # y = Ly
        u[k, 0, :] = phi1(y_grid, t, mu1, mu2) # x = 0
        u[k, -1, :] = phi2(y_grid, t, mu1, mu2) # x = Lx

    for k in range(Nt):
        tmp = deepcopy(u[k])
        t_half = (k + 0.5) * tau
        t_next = (k + 1) * tau

        # Первый дробный шаг (по x)
        for j in range(1, Ny):
            A = np.zeros((Nx - 1, Nx - 1))
            d = np.zeros(Nx - 1)
            sigma_x = a * tau / (2 * hx * hx)
            sigma_y = a * tau / (2 * hy * hy)

            # Граничные узлы
            A[0, 0] = 1 + 2 * sigma_x
            if Nx > 2:
                A[0, 1] = -sigma_x
            d[0] = (u[k, 1, j] +
                    sigma_y * (u[k, 1, j - 1] - 2 * u[k, 1, j] + u[k,
1, j + 1])) +
                    sigma_x * phi1(y_grid[j], t_half, mu1, mu2))

            for i in range(1, Nx - 2):
                A[i, i - 1] = -sigma_x
                A[i, i] = 1 + 2 * sigma_x
                A[i, i + 1] = -sigma_x
                d[i] = (u[k, i + 1, j] +
                        sigma_y * (u[k, i + 1, j - 1] - 2 * u[k, i +
1, j] + u[k, i + 1, j + 1])) +
                        sigma_x * phi2(y_grid[j], t_half, mu1, mu2))

            if Nx > 2:
                A[-1, -2] = -sigma_x
                A[-1, -1] = 1 + 2 * sigma_x
                d[-1] = (u[k, -2, j] +
                        sigma_y * (u[k, -2, j - 1] - 2 * u[k, -2, j]
+ u[k, -2, j + 1])) +
                        sigma_x * phi2(y_grid[j], t_half, mu1, mu2))

            sol = solve(A, d)
            tmp[1:-1, j] = sol

        # Обновим границы tmp
        tmp[:, 0] = phi3(x_grid, t_half, mu1, mu2)
        tmp[:, -1] = phi4(x_grid, t_half, mu1, mu2)

```

```

tmp[0, :] = phi1(y_grid, t_half, mu1, mu2)
tmp[-1, :] = phi2(y_grid, t_half, mu1, mu2)

# Второй дробный шаг (по y)
for i in range(1, Nx):
    A = np.zeros((Ny - 1, Ny - 1))
    d = np.zeros(Ny - 1)
    sigma_x = a * tau / (2 * hx * hx)
    sigma_y = a * tau / (2 * hy * hy)

    A[0, 0] = 1 + 2 * sigma_y
    if Ny > 2:
        A[0, 1] = -sigma_y
    d[0] = (tmp[i, 1] +
            sigma_x * (tmp[i - 1, 1] - 2 * tmp[i, 1] + tmp[i +
1, 1])) +
            sigma_y * phi3(x_grid[i], t_next, mu1, mu2))

    for j in range(1, Ny - 2):
        A[j, j - 1] = -sigma_y
        A[j, j] = 1 + 2 * sigma_y
        A[j, j + 1] = -sigma_y
        d[j] = (tmp[i, j + 1] +
                sigma_x * (tmp[i - 1, j + 1] - 2 * tmp[i, j +
1] + tmp[i + 1, j + 1]))

    if Ny > 2:
        A[-1, -2] = -sigma_y
        A[-1, -1] = 1 + 2 * sigma_y
        d[-1] = (tmp[i, -2] +
                sigma_x * (tmp[i - 1, -2] - 2 * tmp[i, -2] +
tmp[i + 1, -2])) +
                sigma_y * phi4(x_grid[i], t_next, mu1, mu2))

    sol = solve(A, d)
    u[k + 1, i, 1:-1] = sol

# Обновим границы u[k+1]
u[k + 1, :, 0] = phi3(x_grid, t_next, mu1, mu2)
u[k + 1, :, -1] = phi4(x_grid, t_next, mu1, mu2)
u[k + 1, 0, :] = phi1(y_grid, t_next, mu1, mu2)
u[k + 1, -1, :] = phi2(y_grid, t_next, mu1, mu2)

return u

```

```

def FractionalStepsMethod(Nt, Nx, Ny, tau, hx, hy, mu1, mu2):
    Lx = mu1 * np.pi / 2
    Ly = mu2 * np.pi / 2
    u = np.zeros((Nt + 1, Nx + 1, Ny + 1))
    x_grid = np.linspace(0, Lx, Nx + 1)
    y_grid = np.linspace(0, Ly, Ny + 1)

    for i in range(Nx + 1):
        for j in range(Ny + 1):
            u[0, i, j] = psi(x_grid[i], y_grid[j], mu1, mu2)

    for k in range(Nt + 1):
        t = k * tau
        u[k, :, 0] = phi3(x_grid, t, mu1, mu2)
        u[k, :, -1] = phi4(x_grid, t, mu1, mu2)
        u[k, 0, :] = phi1(y_grid, t, mu1, mu2)
        u[k, -1, :] = phi2(y_grid, t, mu1, mu2)

    for k in range(Nt):
        tmp = deepcopy(u[k])

```

```

t_half = (k + 0.5) * tau
t_next = (k + 1) * tau

# Первый шаг (по x)
r = a * tau / (hx * hx)
for j in range(1, Ny):
    A = np.zeros((Nx - 1, Nx - 1))
    d = np.zeros(Nx - 1)
    A[0, 0] = 1 + 2 * r
    if Nx > 2:
        A[0, 1] = -r
    d[0] = u[k, 1, j] + r * phi1(y_grid[j], t_half, mu1, mu2)

    for i in range(1, Nx - 2):
        A[i, i - 1] = -r
        A[i, i] = 1 + 2 * r
        A[i, i + 1] = -r
        d[i] = u[k, i + 1, j]

    if Nx > 2:
        A[-1, -2] = -r
        A[-1, -1] = 1 + 2 * r
        d[-1] = u[k, -2, j] + r * phi2(y_grid[j], t_half, mu1,
mu2)

    sol = solve(A, d)
    tmp[1:-1, j] = sol

tmp[:, 0] = phi3(x_grid, t_half, mu1, mu2)
tmp[:, -1] = phi4(x_grid, t_half, mu1, mu2)
tmp[0, :] = phi1(y_grid, t_half, mu1, mu2)
tmp[-1, :] = phi2(y_grid, t_half, mu1, mu2)

# Второй шаг (по y)
r = a * tau / (hy * hy)
for i in range(1, Nx):
    A = np.zeros((Ny - 1, Ny - 1))
    d = np.zeros(Ny - 1)
    A[0, 0] = 1 + 2 * r
    if Ny > 2:
        A[0, 1] = -r
    d[0] = tmp[i, 1] + r * phi3(x_grid[i], t_next, mu1, mu2)

    for j in range(1, Ny - 2):
        A[j, j - 1] = -r
        A[j, j] = 1 + 2 * r
        A[j, j + 1] = -r
        d[j] = tmp[i, j + 1]

    if Ny > 2:
        A[-1, -2] = -r
        A[-1, -1] = 1 + 2 * r
        d[-1] = tmp[i, -2] + r * phi4(x_grid[i], t_next, mu1,
mu2)

    sol = solve(A, d)
    u[k + 1, i, 1:-1] = sol

u[k + 1, :, 0] = phi3(x_grid, t_next, mu1, mu2)
u[k + 1, :, -1] = phi4(x_grid, t_next, mu1, mu2)
u[k + 1, 0, :] = phi1(y_grid, t_next, mu1, mu2)
u[k + 1, -1, :] = phi2(y_grid, t_next, mu1, mu2)

return u

```

```

def show_solution(Nx, Ny, Nt, hx, hy, tau, u_adi, u_fs, mu1, mu2):
    Lx = mu1 * np.pi / 2
    Ly = mu2 * np.pi / 2
    x_array = np.linspace(0, Lx, Nx + 1)
    y_array = np.linspace(0, Ly, Ny + 1)

    fig, ax = plt.subplots(2, 3, figsize=(18, 10))
    t_indices = [int(Nt * 0.05), int(Nt * 0.4), int(Nt * 0.7)]
    x_fix = Nx // 2
    y_fix = Ny // 4

    for col, k in enumerate(t_indices):
        t_val = k * tau

        # === Сечение по x (фиксированный y) ===
        u_exact_x = np.array([U(x, y_array[y_fix], t_val, mu1, mu2)
        for x in x_array])
        u_adi_x = u_adi[k, :, y_fix]
        u_fs_x = u_fs[k, :, y_fix]

        ax[0, col].plot(x_array, u_exact_x, 'k-', label='Analytical')
        ax[0, col].plot(x_array, u_adi_x, 'b--',
        label='VariableDirections')
        ax[0, col].plot(x_array, u_fs_x, 'r-.',
        label='FractionalSteps')
        ax[0, col].set_xlabel('x')
        ax[0, col].set_ylabel('U(x, y, t)')
        ax[0, col].set_title(f'x (y={y_array[y_fix]:.2f})\n t =
        {t_val:.3f}')
        ax[0, col].grid(True)
        ax[0, col].legend()

        # === Сечение по y (фиксированный x) ===
        u_exact_y = np.array([U(x_array[x_fix], y, t_val, mu1, mu2)
        for y in y_array])
        u_adi_y = u_adi[k, x_fix, :]
        u_fs_y = u_fs[k, x_fix, :]

        ax[1, col].plot(y_array, u_exact_y, 'k-', label='Analytical')
        ax[1, col].plot(y_array, u_adi_y, 'b--',
        label='VariableDirections')
        ax[1, col].plot(y_array, u_fs_y, 'r-.',
        label='FractionalSteps')
        ax[1, col].set_xlabel('y')
        ax[1, col].set_ylabel('U(x, y, t)')
        ax[1, col].set_title(f'y (x={x_array[x_fix]:.2f})\n t =
        {t_val:.3f}')
        ax[1, col].grid(True)
        ax[1, col].legend()

    plt.tight_layout()
    plt.show()

def error(Nt, Lx, Ly, tau, mu1, mu2):
    N_array = [10, 20, 40]
    errors1x = []
    errors2x = []
    for N in N_array:
        hx = Lx / N
        hy = Ly / N
        # Методы
        u1 = VariableDirectionMethod(Nt, N, N, tau, hx, hy, mu1, mu2)
        u2 = FractionalStepsMethod(Nt, N, N, tau, hx, hy, mu1, mu2)
        # Точка для ошибки
        t = tau * Nt / 2

```



```

        x_mid = Lx / 2
        y_mid = Ly / 2
        # Сечение по x
        ux_exact = np.array([U(x_i * hx, y_mid, t, mu1, mu2) for x_i
in range(N + 1)])
        u1x = u1[int(Nt / 2), :, int(N / 2)]
        u2x = u2[int(Nt / 2), :, int(N / 2)]
        errors1x.append(np.max(np.abs(ux_exact - u1x)))
        errors2x.append(np.max(np.abs(ux_exact - u2x)))
    return N_array, np.array(errors1x), np.array(errors2x)

def show_errors(Nt, Lx, Ly, tau, mu1, mu2):
    N_array, errors1x, errors2x = error(Nt, Lx, Ly, tau, mu1, mu2)
    delta_x = np.array([Lx / N for N in N_array])
    plt.figure()
    plt.plot(delta_x, errors1x, 'b-', label='VariableDirections')
    plt.plot(delta_x, errors2x, 'r--', label='FractionalSteps')
    plt.xlabel('delta x')
    plt.ylabel('Epsilon')
    plt.title('Error')
    plt.grid()
    plt.legend()
    plt.show()

def plot_3d_solution(x_grid, y_grid, u_adi, u_fs, mu1, mu2, T,
case_name):
    from mpl_toolkits.mplot3d import Axes3D # noqa: F401

    X, Y = np.meshgrid(x_grid, y_grid, indexing='ij')

    # Аналитическое решение в момент времени T
    U_anal = np.array([U(x, y, T, mu1, mu2) for y in y_grid] for x in
x_grid])
    U_adi = u_adi[-1] # Последний временной слой
    U_fs = u_fs[-1]

    fig = plt.figure(figsize=(18, 5))

    # Аналитическое
    ax1 = fig.add_subplot(1, 3, 1, projection='3d')
    surf1 = ax1.plot_surface(X, Y, U_anal, cmap='viridis', alpha=0.9)
    ax1.set_title(f'{case_name}\nAnalytical')
    ax1.set_xlabel('x'); ax1.set_ylabel('y'); ax1.set_zlabel('u')
    fig.colorbar(surf1, ax=ax1, shrink=0.5)

    # Метод переменных направлений (ADI)
    ax2 = fig.add_subplot(1, 3, 2, projection='3d')
    surf2 = ax2.plot_surface(X, Y, U_adi, cmap='viridis', alpha=0.9)
    ax2.set_title(f'{case_name}\nVariableDirections')
    ax2.set_xlabel('x'); ax2.set_ylabel('y'); ax2.set_zlabel('u')
    fig.colorbar(surf2, ax=ax2, shrink=0.5)

    # Метод дробных шагов (Fractional Steps)
    ax3 = fig.add_subplot(1, 3, 3, projection='3d')
    surf3 = ax3.plot_surface(X, Y, U_fs, cmap='viridis', alpha=0.9)
    ax3.set_title(f'{case_name}\nFractionalSteps')
    ax3.set_xlabel('x'); ax3.set_ylabel('y'); ax3.set_zlabel('u')
    fig.colorbar(surf3, ax=ax3, shrink=0.5)

    plt.tight_layout()
    plt.show()

def main():

```

```

cases = [
    (1, 1, "μ1=1, μ2=1"),
    (2, 1, "μ1=2, μ2=1"),
    (1, 2, "μ1=1, μ2=2")
]

Nx = Ny = 51
Nt = 50
T = 0.1

for mu1, mu2, case_name in cases:
    print(f"Случай: {case_name}\n")

    Lx = mu1 * np.pi / 2
    Ly = mu2 * np.pi / 2
    hx = Lx / Nx
    hy = Ly / Ny
    tau = T / Nt

    u_adi = VariableDirectionMethod(Nt, Nx, Ny, tau, hx, hy, mu1,
mu2)
    u_fs = FractionalStepsMethod(Nt, Nx, Ny, tau, hx, hy, mu1,
mu2)

    show_solution(Nx, Ny, Nt, hx, hy, tau, u_adi, u_fs, mu1, mu2)
    show_errors(Nt, Lx, Ly, tau, mu1, mu2)

    x_grid = np.linspace(0, Lx, Nx + 1)
    y_grid = np.linspace(0, Ly, Ny + 1)
    plot_3d_solution(x_grid, y_grid, u_adi, u_fs, mu1, mu2, T,
case_name)

if __name__ == "__main__":
    main()

```

Результат работы программы:

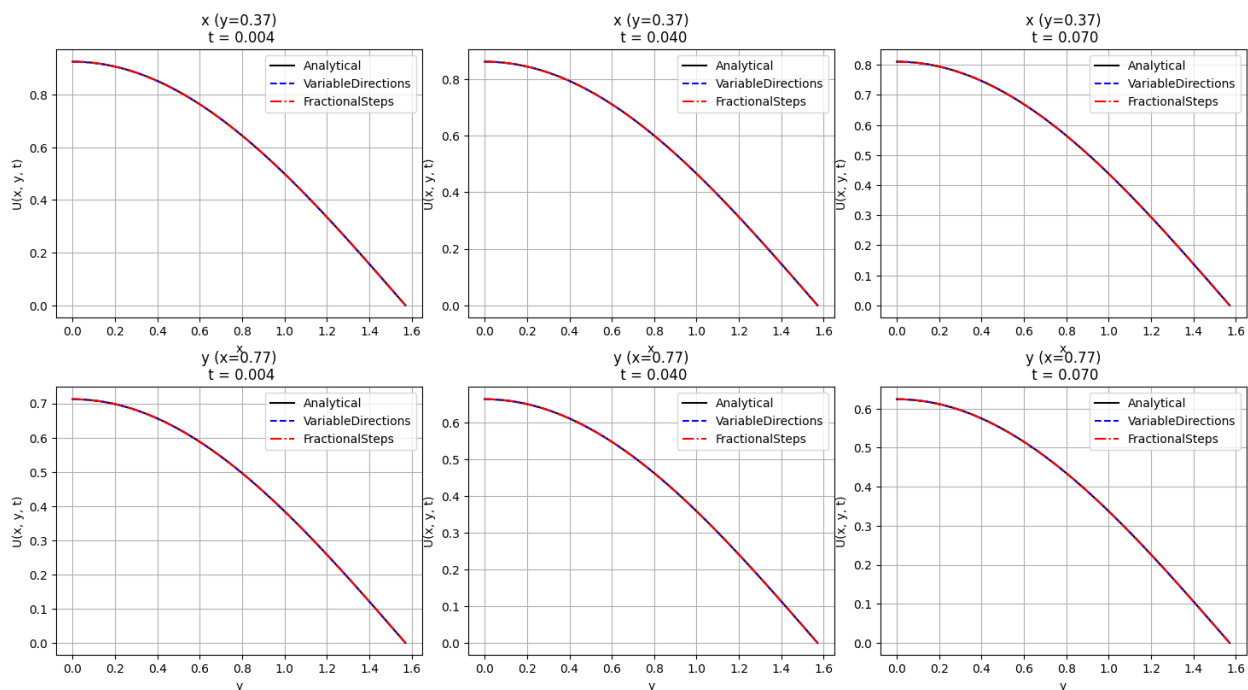


Рисунок 27. Сравнение численных решений с аналитическим в различные моменты времени ($\mu_1 = 1$, $\mu_2 = 1$)

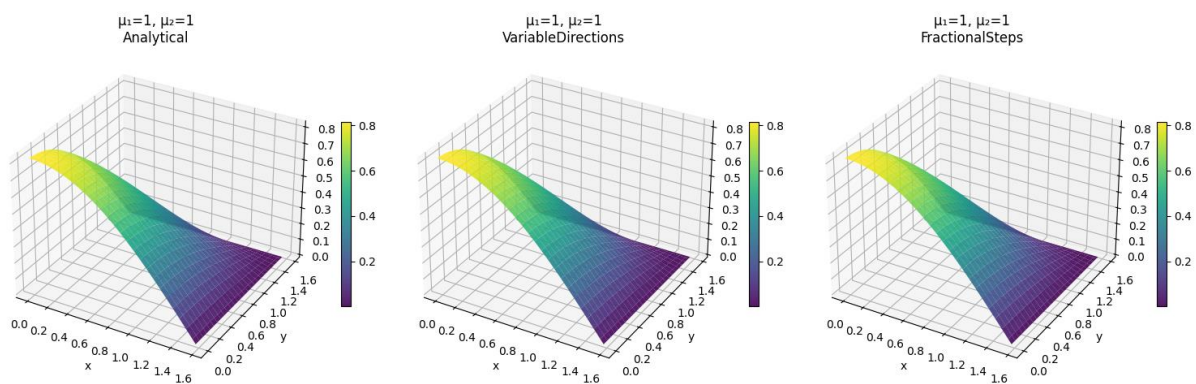


Рисунок 28. Трехмерная визуализация сравнения методов ($\mu_1 = 1$, $\mu_2 = 1$)

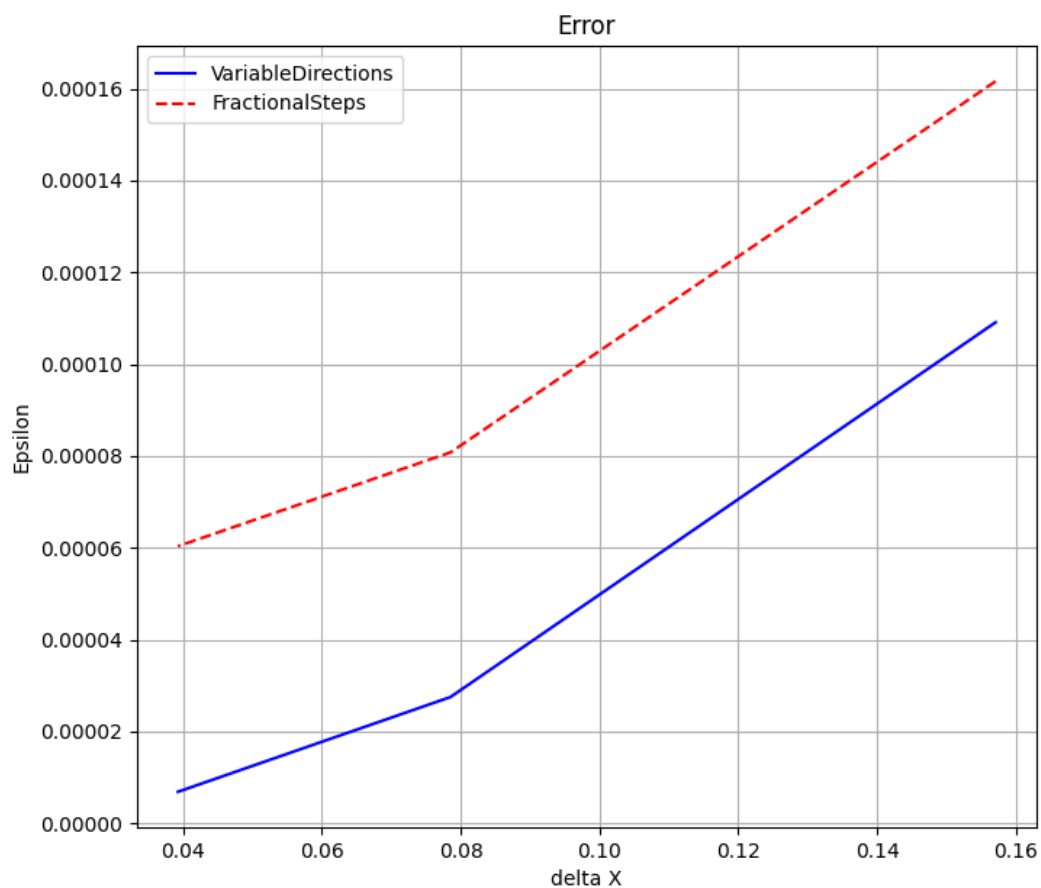


Рисунок 29. Погрешности методов ($\mu_1 = 1, \mu_2 = 1$)

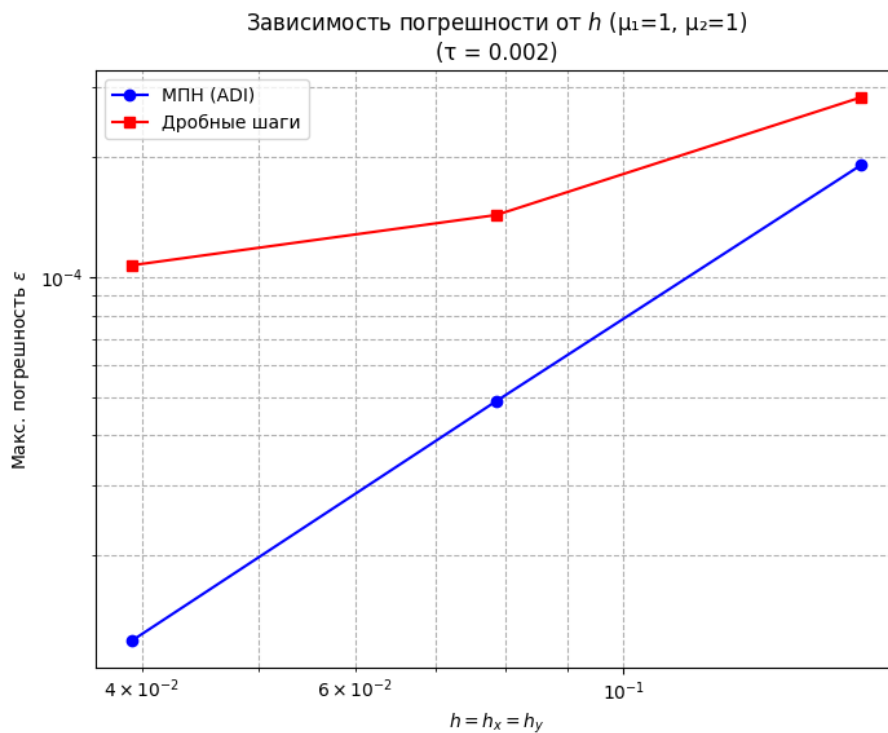


Рисунок 30. Погрешность от шага по пространству ($\mu_1 = 1, \mu_2 = 1$)

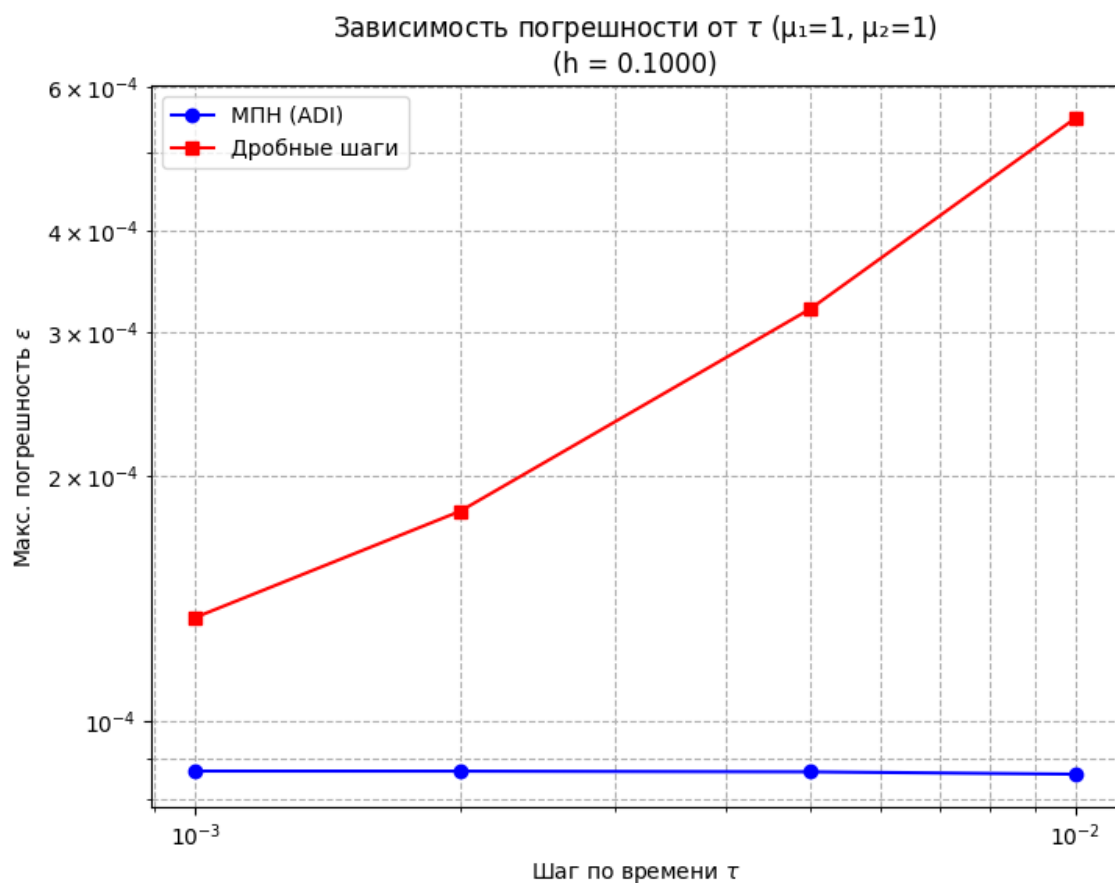


Рисунок 31. Погрешность от шага по времени ($\mu_1 = 1, \mu_2 = 1$)

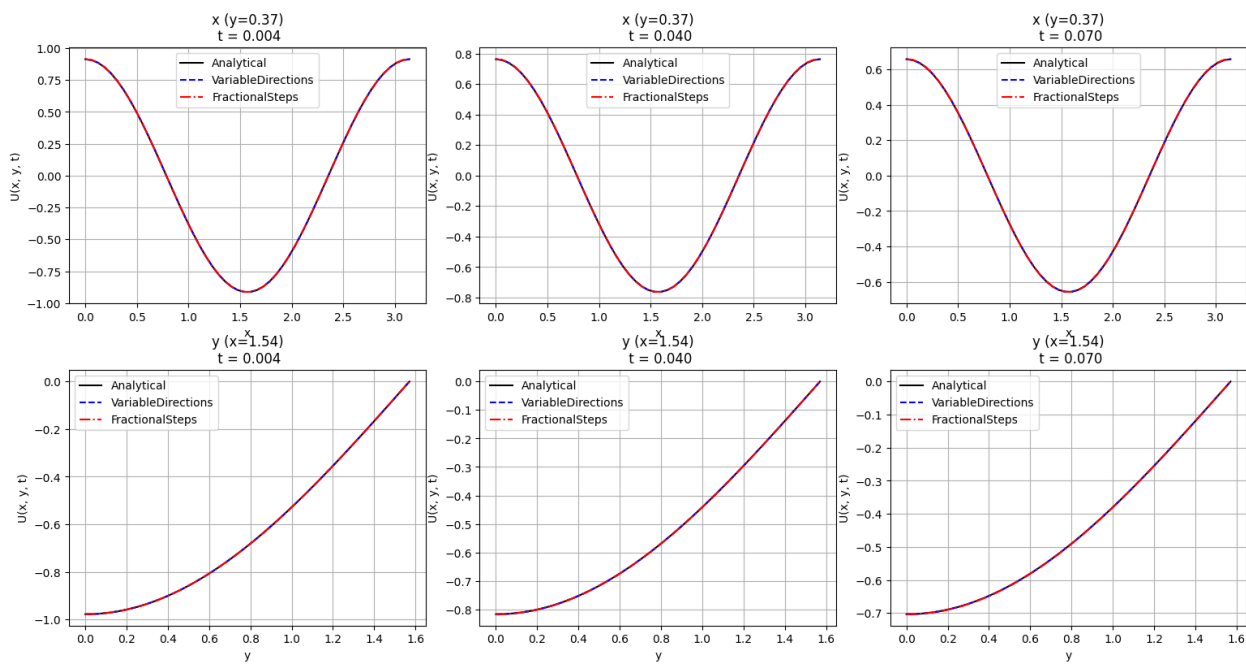


Рисунок 32. Сравнение численных решений с аналитическим в различные моменты времени ($\mu_1 = 2, \mu_2 = 1$)

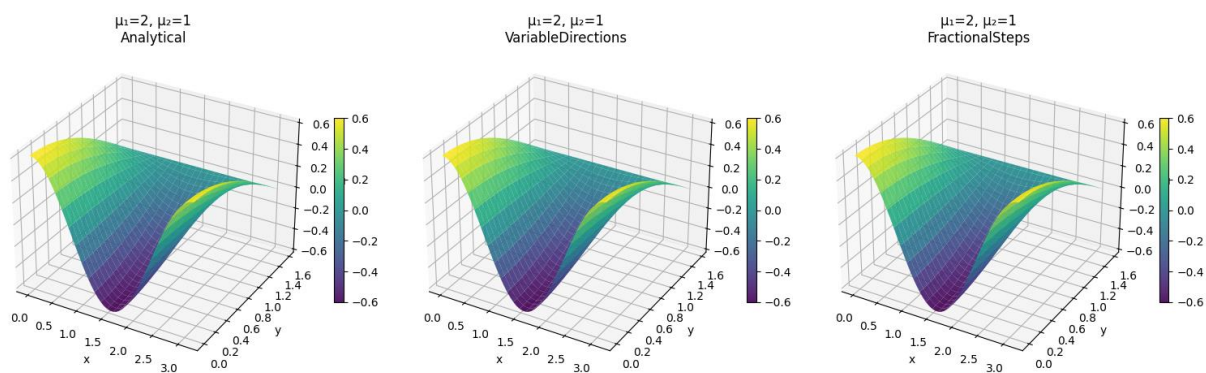


Рисунок 33. Трехмерная визуализация сравнения методов ($\mu_1 = 2$, $\mu_2 = 1$)

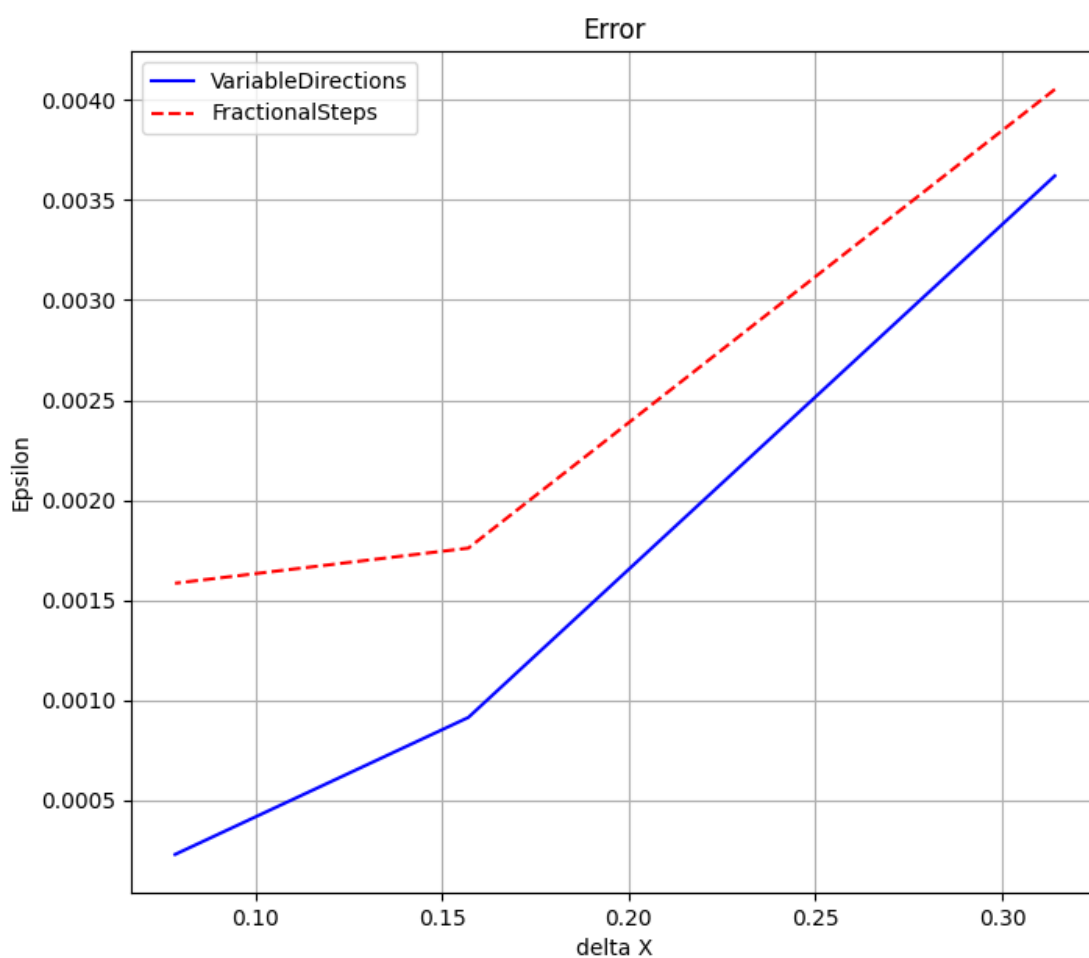


Рисунок 34. Погрешности методов ($\mu_1 = 2$, $\mu_2 = 1$)

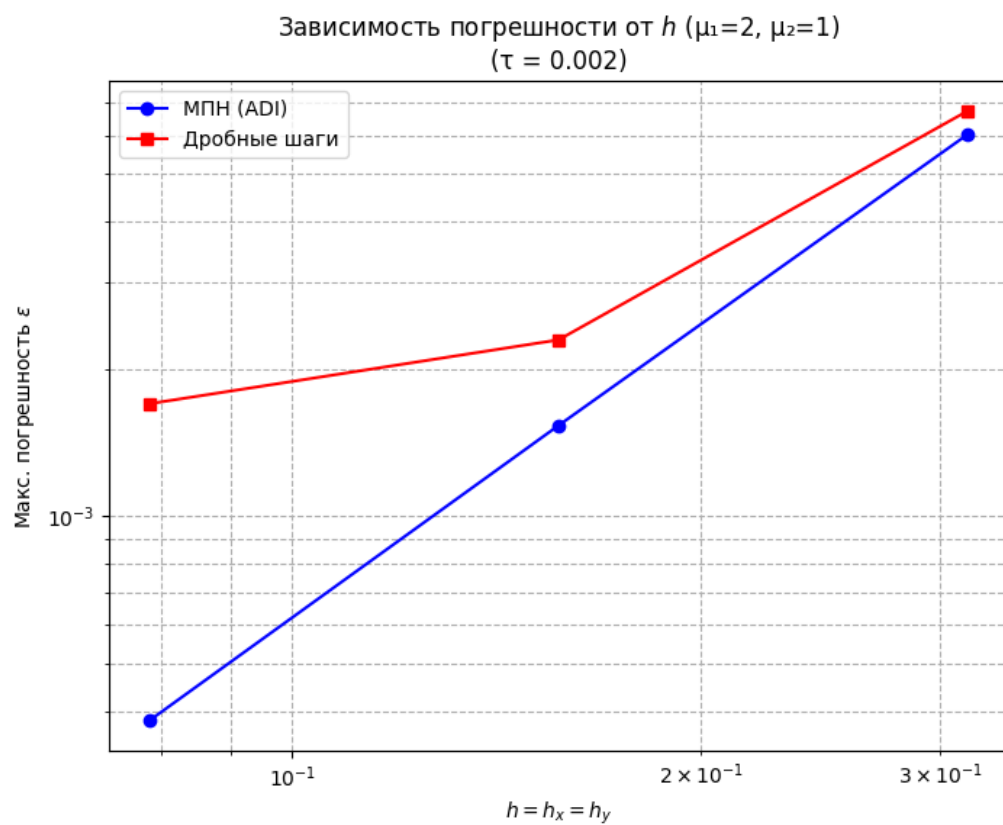


Рисунок 35. Погрешность от шага по пространству ($\mu_1 = 2, \mu_2 = 1$)

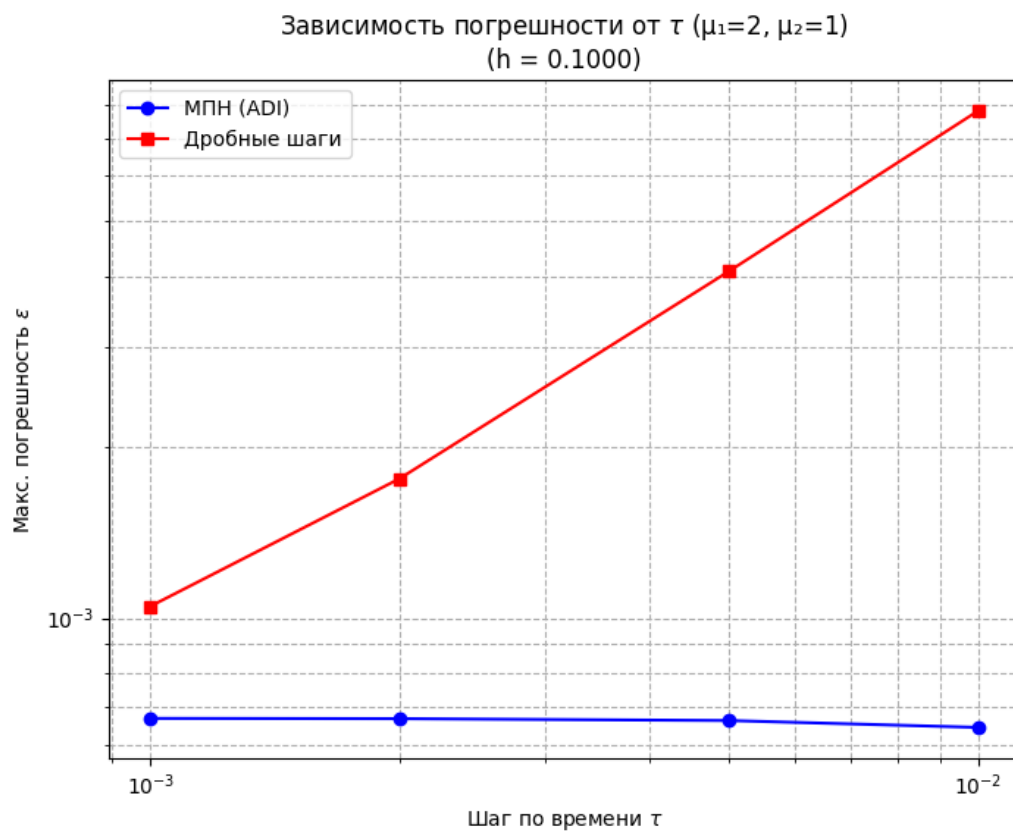


Рисунок 36. Погрешность от шага по времени ($\mu_1 = 2, \mu_2 = 1$)

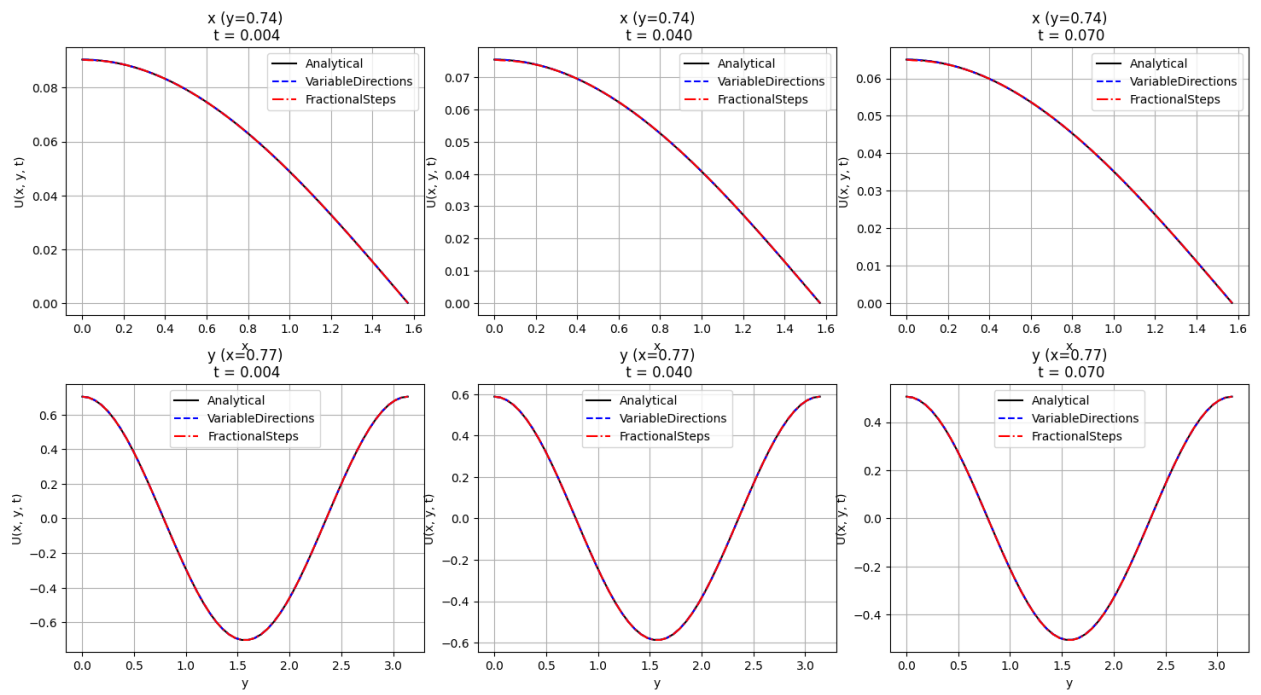


Рисунок 37. Сравнение численных решений с аналитическим в различные моменты времени ($\mu_1 = 1$, $\mu_2 = 2$)

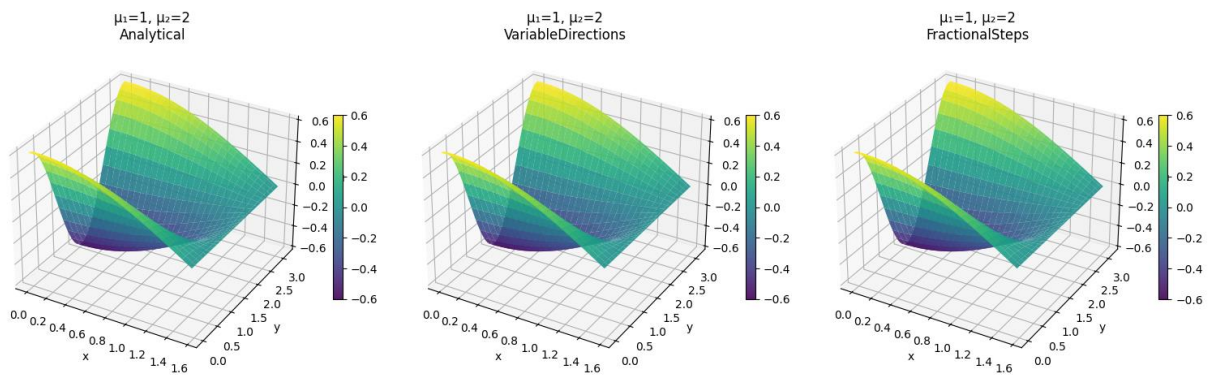


Рисунок 38. Трехмерная визуализация сравнения методов ($\mu_1 = 1$, $\mu_2 = 2$)

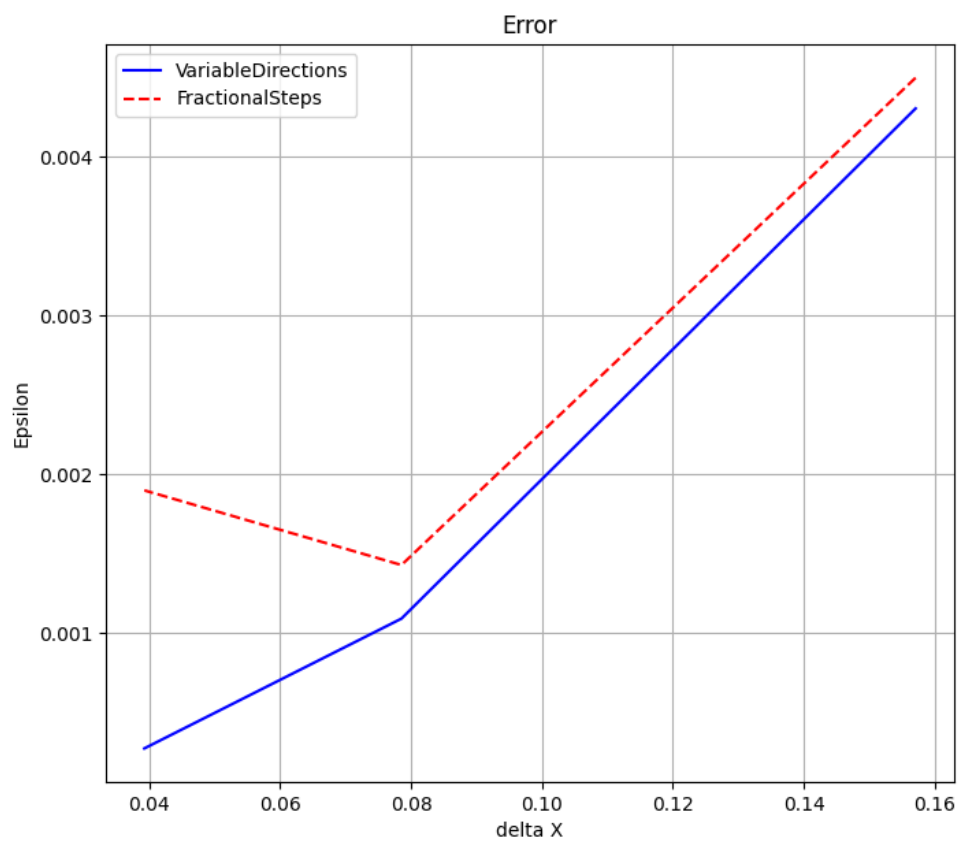


Рисунок 39. Погрешности методов ($\mu_1 = 1$, $\mu_2 = 2$)

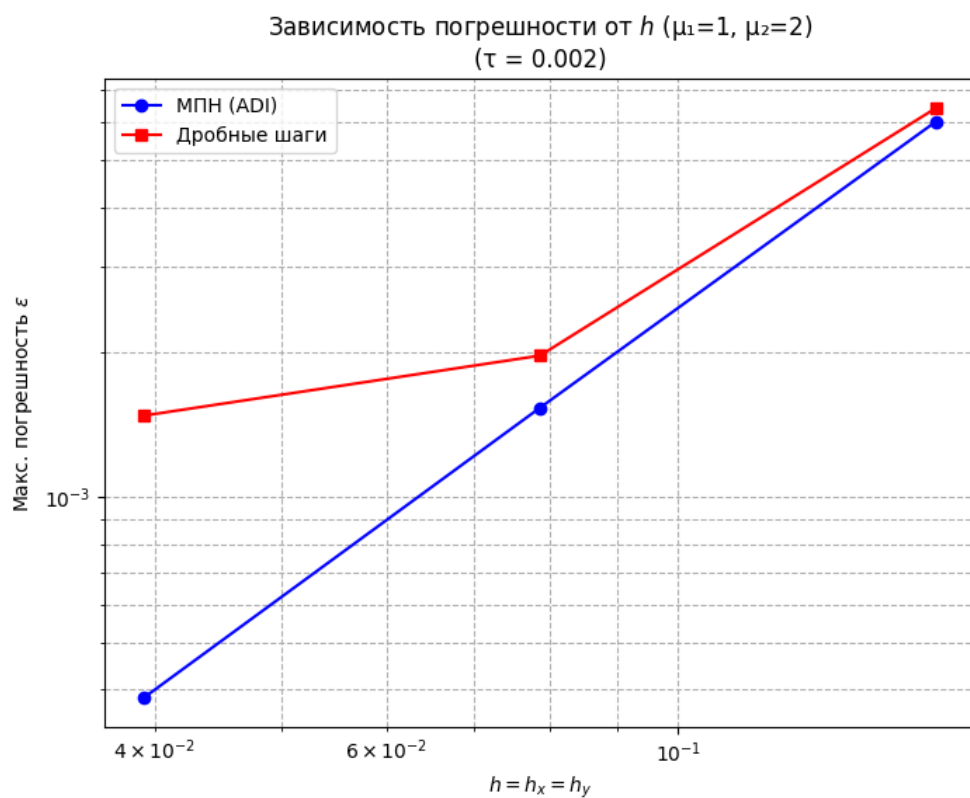


Рисунок 40. Погрешность от шага по пространству ($\mu_1 = 1$, $\mu_2 = 2$)

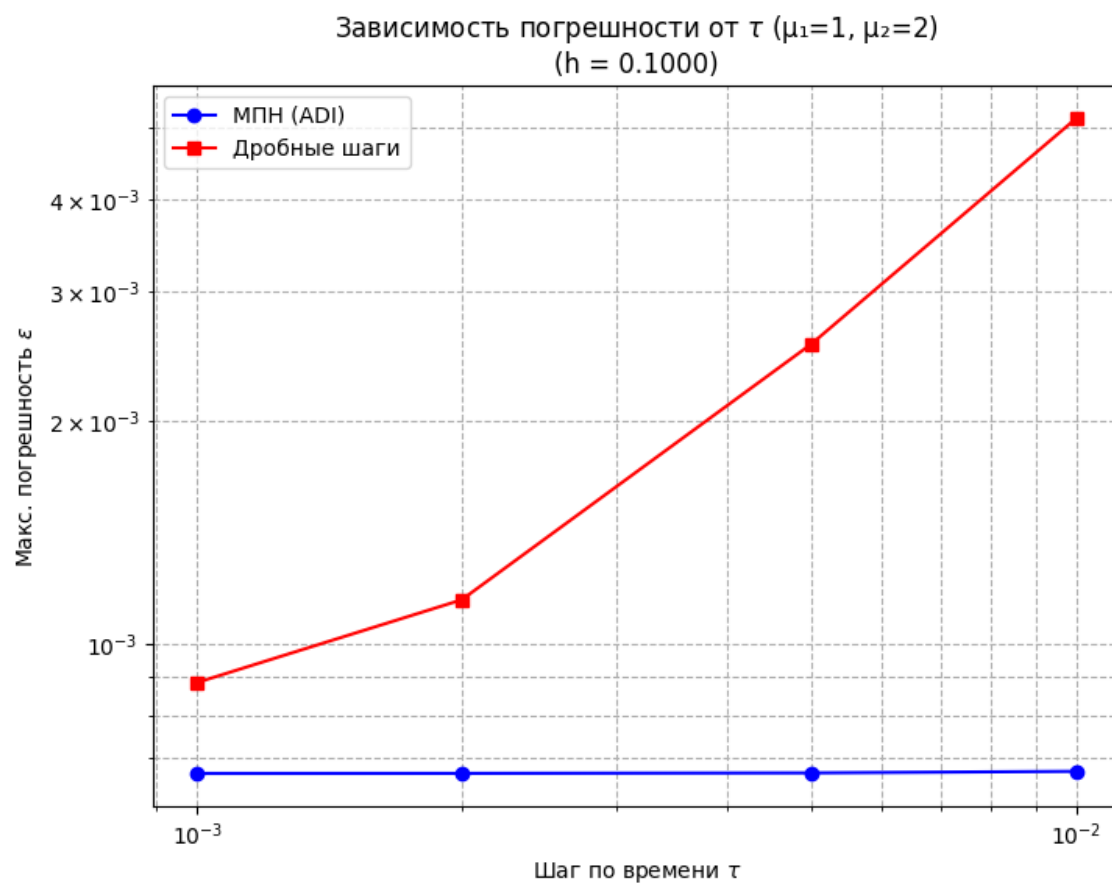


Рисунок 41. Погрешность от шага по времени ($\mu_1 = 1, \mu_2 = 2$)

Выводы:

Было написано несколько программ на Python для численного моделирования дифференциальных уравнений с частными производными. Были решены одномерные начально-краевые задачи для дифференциальных уравнений параболического, гиперболического, эллиптического типов и двумерная начально-краевая задачу для дифференциального уравнения параболического типа.

В ходе выполнения данных лабораторных работ были сделаны ценные теоретические и практические выводы. Было изучено множество методов для решения поставленных задач, которые впоследствии были применены на практике.

В результате выполнения данных лабораторных работ были приобретены навыки, которые будут полезны для выполнения других работ и курсовых проектов.